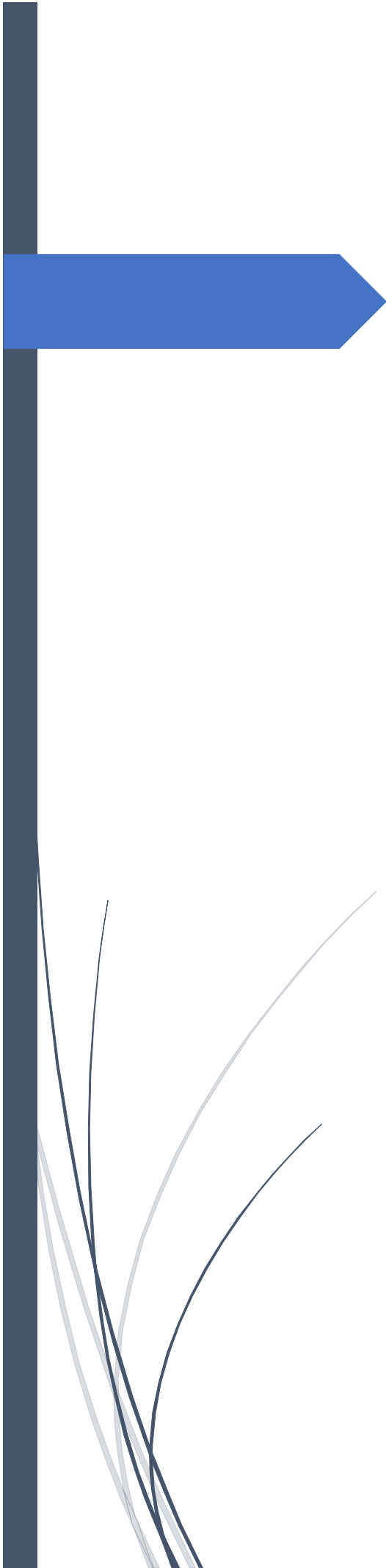




Elementi di Programmazione 1

Gabriel Rovesti

Attenzione



Il file non ha alcuna pretesa di correttezza; di fatto, è una riscrittura attenta di appunti, slide, materiale sparso in rete, approfondimenti personali dettagliati al meglio delle mie capacità. Credo comunque che, per scopo didattico e di piacere di imparare (sì, io studio per quello e non solo per l'esame) questo file possa essere utile. Semplice si pone, per davvero ci prova.

Thank me sometimes, it won't kill you that much.

Gabriel

Sommario

Introduzione	3
Comandi di base	5
Correttezza	10
Funzioni	11
Tipi	14
Array	16
Puntatori.....	18
Correttezza: PRE e POST Condizioni	25
Ricorsione: tipi di ricorsione, correttezza ed induzione	28
File multipli, inclusione, variabili globali, variabili statiche ed esercizi	34
Cenni di complessità computazionale	43
Struct ed union, enum, file in C, operazioni sui file/file binari, allocazione della memoria.....	46
Liste collegate con puntatori (linked lists), alberi, alberi binari di ricerca	59



Introduzione

Viene definito algoritmo un insieme di istruzioni ordinate e finite di istruzioni elementari, chiare e non ambigue, per risolvere un problema. Il concetto di algoritmo è generale (non ci sono riferimenti al calcolatore). Di fatto, si hanno:

- 3 elementi
 - o Un problema da risolvere
 - o Una sequenza di istruzioni
 - o Un esecutore di istruzioni
- 2 attori
 - o Chi crea le istruzioni
 - o Chi esegue le istruzioni

La risoluzione di un problema spesso implica la risoluzione di una serie di sottoproblemi e spesso significa seguire le istruzioni in modo chiaro, preciso ed ordinato.

Programmare significa scrivere algoritmi per il calcolatore

Data la descrizione di un problema da risolvere

1. Si descrive l'algoritmo di risoluzione
2. Si traduce l'algoritmo nel linguaggio di programmazione
 - più adatto oppure in quello imposto dal problema
3. Si fornisce prova che il nostro programma fa quello richiesto
 - Il tipo di prova dipende dal contesto
 - unit tests
 - prove di correttezza

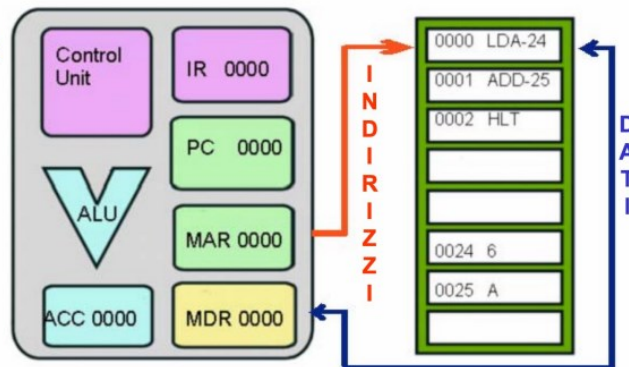
A parità di correttezza, come si può migliorare un programma?

1. Efficienza (tempo e spazio), sia a livello algoritmico che implementativo
 - a. Il secondo aspetto è preponderante nel corso, ma tratteremo entrambi
 - b. Cenni di complessità
2. Organizzazione: si codifica (per bene) una volta, poi si riusa il codice
 - a. Si divide il problema in sottoproblemi e si cerca di riusare le soluzioni già implementate in passato
3. Stile
 - a. Il codice deve essere comprensibile ai tuoi colleghi e a te stesso tra 3 mesi.

Notiamo in particolare le specifiche del Linguaggio Macchina.

- Il Computer è capace di eseguire programmi in linguaggio macchina, tramite il ciclo FDE

- [F] - estrae una istruzione dalla memoria (in codice binario)
- [D] - individua i circuiti corrispondenti all'istruzione
- [E] - attiva tali circuiti



Il linguaggio della macchina è un linguaggio di basso livello (il livello di dettaglio tende a far perdere la visione d'insieme), poiché dipendente dall'architettura.

- Se cambia architettura, dobbiamo aggiornare il programma.
- Difficile da ricordare (sequenze di 0 e 1)
 - o Soluzione: Il linguaggio assembler è una versione mnemonica del linguaggio macchina, es. ADD invece della sequenza di bit dell'addizione
 - o l'assemblatore sostituisce le istruzioni con i corrispondenti codici macchina

Ecco quindi che si fornisce uno strumento che traduca i nostri programmi nel linguaggio della macchina ospite: il traduttore. Distinguiamo le due seguenti nozioni:

- **Interprete**: traduce una istruzione di alto livello e la esegue immediatamente
 - o più lenta l'esecuzione del programma
 - o necessita del traduttore per eseguire il programma
 - o se si ha il traduttore ed il codice sorgente, può essere eseguito su ogni computer
- **Compilatore**: traduce tutte le istruzioni assieme che vengono poi eseguite tutte assieme direttamente in linguaggio macchina (C, C++)
 - o più lenta l'esecuzione del programma
 - o necessita del traduttore per eseguire il programma
 - o se si ha il traduttore ed il codice sorgente, può essere eseguito su ogni computer

Possono tuttavia esistere soluzioni intermedie: compilazione in bytecode ed interpretazione (Java).

Nel corso viene usato un linguaggio semplice: il C. Esso è linguaggio compilato; insieme ristretto di comandi di base (ci si affida a librerie di funzioni), il compilatore è "facile" da scrivere, quindi portabilità.

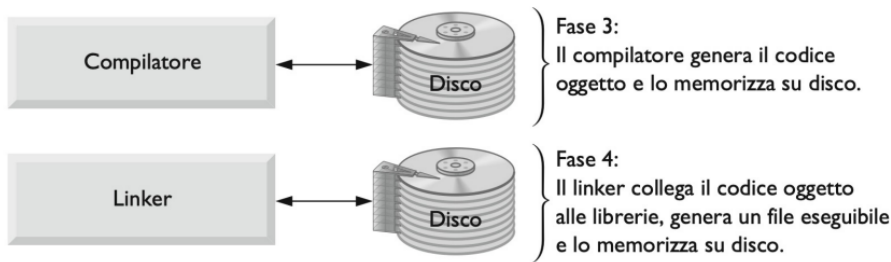
La fase di traduzione avviene graficamente come segue:



```
#include <stdio.h>
int main (void) {
    printf("Ciao Mondo!\n"); //stampa il messaggio Ciao Mondo!
}
```



- Rimozione dei commenti
- `#include <x>`: il contenuto del file x viene copiato all'inizio del nostro file (x contiene informazioni su come eseguire comandi aggiuntivi, ad esempio `stdio.h` permette di utilizzare il comando `printf`)
- Espansione delle macro (le vedremo tra qualche lezione)
- Compilazione condizionale (utile se alcune librerie hanno nomi diversi in diversi sistemi operativi)



Un programma è generalmente composto da molti file ed utilizza funzioni già scritte da altri (printf). Per evitare di duplicare il codice di tali funzioni, si caricano in memoria una volta e si collegano al nostro programma (linking)

```
#include <stdio.h>

int main(void) {
    printf("Ciao Mondo!\n"); //stampa il messaggio Ciao Mondo!
}
```

- Il Computer è capace di eseguire programmi in linguaggio macchina
- Bisogna tradurre dal linguaggio ad alto livello a quello macchina!
- gcc -o ciao hello_world.c
- Il codice tradotto è a questo punto eseguibile: [su linux] ./ciao

Esiste comunque un ampio insieme di compilatori di C, basati su ANSI C, una serie di specifiche che standardizzano il comportamento del compilatore.

Comandi di base

Il C supporta una serie di operatori, principalmente tutti quelli aritmetici, ma anche quelli condizionali e quelli logici.

Operatore	Significato	Esempio
==	uguale a	5 == 3 è Falso
>	maggiore di	5 > 3 è Vero
<	minore di	5 < 3 è Falso
!=	diverso da	5 != 3 è Vero
>=	maggiore o uguale di	5 >= 3 è Vero
<=	minore o uguale di	5 <= 3 è Falso
&& (binario)	AND logico. Vero solo se entrambi gli operandi sono Veri	((5==5) && (5<2)) è Falso.
(binario)	OR logico. Vero se almeno uno degli argomenti è Vero	((5==5) (5<2)) è Vero
! (unario)	NOT logico. Vero se l'argomento è Falso	!(5==5) è Falso

Si noti che:

- ! ha precedenza su &&, ||, che si applicano da sinistra a destra (come per le espressioni si usano le parentesi per indicare ordini di precedenza diversi)

Come in matematica, le espressioni e le condizioni possono essere generalizzate utilizzando simboli (variabili) al posto di alcuni valori.

Una variabile ha i seguenti attributi:

- il nome (che definiamo noi)
- l'area di memoria in cui è mantenuto il suo valore (non assegnata dall'utente)
- il tipo: le variabili vengono usate per rappresentare numeri interi, reali, caratteri (in C è definito dall'utente)
 - o Il tipo è un modo conciso per dire quanta memoria occupa (dipende dall'architettura della macchina), come leggere o scrivere la sequenza di bit e quali operazioni posso fare con quella variabile.

Le variabili posseggono due vie di assegnamento (cioè, valore corrente che hanno, distinguendo così):

nome e tipo	L-valore Identificativo dell' area di memoria riservata alla variabile	R-valore il contenuto corrente della cella di memoria
-------------	---	--

0	
1	2
2	4
3	

x: L-valore=2
R-valore=4

- $y = 2;$
- Notate che l'attributo selezionato della variabile (L o R valore) dipende da dove essa compare nell'istruzione:
- $x = y + 2;$ // vai alla cella di memoria indicata dall' L-valore di y e scrivici dentro il risultato dell'espressione alla destra dell'uguale, ovvero il risultato della somma tra 2 e l'R-valore della variabile y: $x=2+2=4$
- $x = x + 1$ // $x=4+1=5$

Le variabili, per poter essere usate, vanno *dichiarate*.

`int x;` // dichiara una variabile di tipo intero

`int x = 2;` // dichiara una variabile di tipo intero ed inizializza il suo valore a 2

Un legame tra una variabile ed un suo attributo si dice statico se è stabilito prima dell'esecuzione e non può essere cambiato in seguito, dinamico altrimenti:

- il valore è un legame dinamico
- In C il tipo è un legame statico (questo implica che il compilatore può identificare i seguenti tipi di errore: `int x; x = "Ciao Mondo!";`);

In C è possibile definire "variabili il cui valore è un legame statico", quelle che comunemente chiamiamo costanti (es. π greco)

- `const int x = 3;` // poiché non possiamo cambiare x, dobbiamo definirne il valore quando dichiariamo la variabile

Le variabili possono avere qualsiasi nome; noi usiamo caratteri alfanumerici, ma il nome non deve iniziare con 0-9 e `_` (underscore). Il C è case sensitive (cioè, distingue tra maiuscolo e minuscolo), ma evitiamo di avere variabili di nome VAR e var oppure variabili che assomigliano ad un comando o ad un elemento del linguaggio: IF, INT. I nomi dovrebbero indicare la loro funzione ed il loro contesto (evitando nomi troppo lunghi).

Scrivere il seguente codice C:

```
/*
 * Trasformare il valore in gradi fahrenheit della variabile fahr (X) nel
 * corrispondente valore celsius (Y) arrotondato all'intero inferiore e stampare
 * "X gradi fahrenheit corrispondono a Y gradi celsius"
 *
 * Ad esempio se fahr=78 stampa
 * 78 gradi fahrenheit corrispondono a 25 gradi celsius
 *
 * Si ricorda che  $C = (5/9)(F-32)$ 
 */
```

```
#include <stdio.h>
#include <math.h>

int main() {
    int fahr;
    scanf("%d", &fahr);
    int celsius = floor((5.0 / 9.0) * (fahr - 32));
    printf("%d gradi fahrenheit corrispondono a %d gradi celsius\n", fahr, celsius);
    return 0;
}
```

Si noti l'utilizzo della funzione *floor*, che fondamentalmente arrotonda all'interno più basso per difetto. La struttura è composta dalla funzione *main* (oppure, funzione principale) e uno *scanf* che acquisisce un numero (%d).

In C, la funzione *main* è la funzione di partenza del programma, che viene eseguita al momento del lancio del programma. Il valore di ritorno di *main* indica lo stato di uscita del programma: un valore di 0 significa che il programma è terminato correttamente, mentre un valore non zero indica un errore o una condizione di uscita anomala.

L'uso di *return 0*; alla fine del *main* è una convenzione standard che indica che il programma è terminato correttamente.

Gli *include*, invece, descrivono le librerie incluse → *stdio.h* per l'input/output standard e *math.h*, libreria (collezione di variabili, funzioni, etc.) per le funzioni matematiche.

Proseguiamo con la presenza di condizionali attraverso il meccanismo *if-else* (se-altrimenti).

if (condizione) {

 //comandi da eseguire se la condizione è vera

} else {

 //comandi da eseguire se la condizione è falsa

}

if (x>=0) //non serve { perché abbiamo un solo comando

 printf("positivo");

else

 printf("negativo");

MA

if (condizione1)

 if (condizione2)

 comando1;

else

 comando2;

Senza {} l'else fa riferimento all'if più vicino (condizione2)

Possiamo avere anche varianti:

- `int x = -2, y;`
 - In questa riga di codice viene dichiarata una variabile intera `x` con il valore iniziale di `-2`. In C, un numero intero preceduto da un segno meno (`-`) è un numero negativo.
 - La dichiarazione `int x = -2` dichiara una variabile `x` di tipo intero e la inizializza con il valore `-2`. La dichiarazione `int y;` dichiara una variabile `y` di tipo intero, ma non la inizializza a un valore specifico. Questo valore verrà assegnato più avanti nel codice.
- `y = 3+(x>0?x:-x); // y=5`
 - Il codice assegna il valore "`y`" utilizzando l'operatore ternario "`? :`"
 - L'operatore ternario "`? :`" è un modo compatto di scrivere una semplice espressione if-else. La sintassi generale è la seguente:
 - *espressione_condizionale ? espressione_vera : espressione_falsa*
 - Nel caso specifico, `x>0` è l'espressione condizionale, che verifica se `x` è maggiore di 0. Se la condizione è vera (`x` è effettivamente maggiore di 0), l'espressione vera `x` viene valutata e utilizzata come valore di `y`. Se la condizione è falsa (`x` non è maggiore di 0), l'espressione falsa `-x` viene valutata e utilizzata come valore di `y`.
 - Dato che `x` è uguale a `-2`, la condizione `x>0` è falsa e l'espressione `-x` viene valutata. Poiché `-x` è uguale a `2`, `y` viene assegnato con il valore `3 + 2 = 5`.

Successivamente, abbiamo l'*iterazione*, che continua ad eseguire comandi finché permane una condizione. Questa normalmente si realizza con il ciclo *while* (finché):

```
while (condizione) {
    //comandi da eseguire se la condizione è vera
}
comando2
```

Il comando while:

1. se la condizione è falsa, non esegue i comandi all'interno del blocco e passa a comando2
2. se la condizione è vera, esegue i comandi all'interno del blocco
3. Una volta eseguiti i comandi del blocco, ritorna al punto 1



Le parentesi graffe racchiudono una sequenza (blocco) di comandi. Sono tipicamente utilizzati in combinazione con altri comandi (if e while), ma possono anche apparire da soli.

Le variabili dichiarate all'interno di un blocco sono dette *locali*:

- le variabili locali sono visibili (utilizzabili) solamente all'interno del blocco nel quale sono definite, con la seguente eccezione:

```
{
    int x=2; //chiamiamo x1 questa istanza di x
    {
        int x=3; // da questo momento x1 non è più visibile
    } // x1 è visibile nuovamente
}
```

Blocco di Istruzioni e Visibilità delle Variabili



```
{ // blocco 1
  int x; //x1
}
```

```
{ //blocco 2
  int x; //x2
  int y;
}
```

Posso definire la stessa variabile x in due blocchi diversi ed è come aver definito due variabili diverse (notate che dentro il blocco 2 non posso accedere a x1 e dentro il blocco 1 non posso accedere a x2)

```
// inizializzazione: es. i = 0
while (condizione: es. i<10) {
    //sequenza di comandi
    //assegnamento: es. i = i +1;
}
si può scrivere come:
for(inizializzazione; condizione; assegnamento) {
    //sequenza di comandi;
}
for (int x=1; x <= 3; x=x+1) {
    printf("Ciao Mondo!\n");
}
```

Esercizio

```
/*
 * Scrivere un programma che stampi x volte "Ciao Mondo!"
 * Es. se x = 3
 * Ciao Mondo!
 * Ciao Mondo!
 * Ciao Mondo!
 */

/*
 * Calcolare la somma dei primi n numeri naturali e stamparla a video
 * Ad es. se n=4 stampa
 * 10
 */
```

Un modo efficiente per calcolare la somma dei primi n numeri naturali è utilizzare la formula matematica per calcolare la somma di una progressione aritmetica. La somma dei primi n numeri naturali è data da $n * (n + 1) / 2$.

Ecco un esempio di codice che implementa questa formula:

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int sum = n * (n + 1) / 2;
    printf("%d\n", sum);
    return 0;
}
```

Questo codice legge n dall'input dell'utente e calcola la somma utilizzando la formula e la stampa a video. L'operazione di divisione intera (/) produce un risultato intero, anche se i due operandi sono numeri a virgola mobile. In questo caso, la divisione intera è sufficiente, poiché la somma dei primi n numeri naturali è sempre un numero intero.

Nella programmazione ci possono essere varie fonti di errore, dovute alla sintassi (come è scritto il codice, sbagliando a scrivere le parole chiave, istruzioni, etc.) oppure la semantica (significato delle parti di codice).

Abbiamo vari strumenti utili:

- Debug: permette di trovare gli errori di implementazione
- Unit testing : può aiutare per individuare errori logici
- Correttezza: aiuta per dimostrare che il nostro programma fa sempre quello che vogliamo

Definiamo quindi:

- Sintassi: insieme di regole che descrive come si possano costruire “frasi” (programmi) validi
- Errore di sintassi: il compilatore riesce a tradurre il nostro codice se e solo se rispetta la sintassi del linguaggio

Se non ci riesce la compilazione fallisce e il file eseguibile non viene generato.

- Il compilatore prova a darci informazioni sull’errore (dove si è bloccato e che tipo di problema ha riscontrato)
- Di solito sono molto precise ed utili, ma a volte (per esempio in casi particolari quando ci si dimentica una parentesi, non indicano dove sia esattamente l’errore)

In aggiunta il compilatore analizza il codice per trovare sequenze di istruzioni possibilmente problematiche anche se corrette sintatticamente, i cosiddetti *warning*.

- Es. una variabile utilizzata prima di essere assegnata
 - o `int x, y; y=x+2; // non si sa quanto valga x!`
- Es. `int x=3; if (x=2) {printf("x=5");}` // un assegnamento ha valore di verità vero!
- Es. `for(i=0; i<10; i=i+1); { printf("%d\n",i); }`
 - o Il blocco viene eseguito una volta stampando 10

Per visualizzare tutti i warning usare:

- `gcc -Wall -o file_eseguibile file_sorgente.c`

Un programma può quindi avere errori semantici (ossia di logica: il programma non fa quello che si voleva) che si evidenziano a run-time (ossia quando si esegue il programma). Il problema può derivare:

- dalla non correttezza dell’algoritmo
- dall’errata implementazione dello stesso

L’analisi di un programma può essere effettuata tramite

1. l’aggiunta di istruzioni printf in opportuni punti del codice che danno informazioni sulle variabili

2. l’uso di un debugger, ossia un programma che

- esegue il codice in modo controllato e permette di:
- eseguire le istruzioni fermandosi dopo ciascuna
- verificare e cambiare il contenuto delle variabili
- fermarsi automaticamente quando si verificano certe condizioni

Come possiamo verificare che il nostro programma calcola ciò che ci aspettiamo?

- Unit tests: calcoliamo a mano una serie di input/output e poi verifichiamo che il calcolatore restituisca, per ogni input, lo stesso output.
 - o Un test fallito dimostra che un programma non è corretto
 - o Una serie di test passati non significa che il programma sia corretto per ogni input, ma soltanto per quelli testati
 - o In certi casi potremmo dover fornire evidenza che, per un certo insieme di input, l’output sia corretto. In questo caso si procede quasi come se fosse una dimostrazione matematica.

Funzioni

Per semplificare la struttura di un programma complesso è possibile suddividerlo in moduli, sottoprogrammi o, nel gergo del C, funzioni. Una funzione è una serie di istruzioni che assolvono a un compito preciso (ad es. calcolare se un numero è primo) e a cui è stato dato un nome.

La sintassi è come segue:

- Sintassi della definizione di una funzione:

```
tipo_restituito nomeFunzione (parametri) {
    //definizioni variabili locali
    comandi della funzione
    return
}
```

La scelta del nome della funzione segue le regole per le variabili

Ogni funzione può essere considerata un piccolo programma isolato dalle altre funzioni.

Una funzione viene invocata scrivendo il suo nome seguito dalle parentesi ()

- nome_funzione()

Definire una funzione è come definire un nuovo comando: dopo che il corpo della funzione è stato eseguito, si torna ad eseguire il comando successivo a nome_funzione().

Una funzione può restituire un valore (per esempio di tipo int), che può essere utilizzato all'interno del codice come se fosse una variabile di quel tipo. Se una funzione non restituisce niente, si usa il tipo void.

```
void stampa_ciao_mondo() {
    int i; // la visibilità di questa funzione è locale al blocco dove è definita
    for(i=0; i<10; i=i+1) {
        printf("ciao mondo!\n");
    }
}
```

```
int main () {
    stampa_ciao_mondo();
    printf("finito\n");
}
```

```
int numero_gatti() {
    return 44; //comando per restituire un valore alla funzione chiamante
}
```

```
int main () {
    int x = numero_gatti();
    printf("Ci sono %d gatti", numero_gatti()+3);
}
```

Vantaggi della programmazione modulare:

- il programma complessivo ha un maggior livello di astrazione perché i moduli “nascondono” al loro interno i dettagli implementativi delle funzionalità realizzate
- il codice per ottenere una certa funzionalità viene scritto una volta sola e viene richiamato ogni volta che è necessario
- il codice complessivo è più corto
- essendo più piccoli, i moduli sono più semplici da implementare e da verificare
- il codice di un modulo correttamente funzionante può essere riutilizzato in altri programmi

Per rendere le funzioni più flessibili ed interessanti, si ha la possibilità di passare, all'interno delle parentesi tonde, dei parametri sui quali la funzione possa operare. Nella definizione della funzione, per ogni parametro bisogna indicare il tipo:

```
int successivo(int n) { //parametro formale della funzione
    return n+1;
}

int main () {
    int x=2;
    printf("x+1=%d\n", successivo(x)); //x=parametro attuale della funzione
}
```

Gli argomenti che la funzione riceve dal chiamante devono essere memorizzati in opportune variabili locali alla funzione stessa dette *parametri*. I parametri sono automaticamente inizializzati con i valori degli argomenti.

- Nell'esempio precedente:

```
int successivo(int n) { return n+1; }
```

quando invoco `successivo(x)`, potete immaginare quello che succede come:

```
int successivo() { int n=x; return n+1; }
```

- in questo caso diciamo che il parametro è passato per valore

Possiamo distinguere i due tipi di parametri:

- I parametri formali sono i parametri dichiarati nella dichiarazione di una funzione o di una procedura. Questi parametri rappresentano le variabili che verranno utilizzate all'interno della funzione per elaborare i dati.
- I parametri attuali sono i valori effettivamente passati alla funzione quando viene chiamata. Questi valori vengono assegnati ai parametri formali all'interno della funzione, che quindi può elaborare i dati.

Ad esempio, consideriamo la seguente funzione:

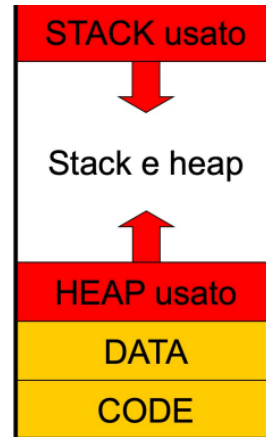
```
void swap(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
}
```

In questo caso, i parametri formali sono `x` e `y`, che rappresentano le variabili utilizzate all'interno della funzione per scambiare i valori. Se la funzione viene chiamata con i valori 1 e 2, i parametri attuali sono 1 e 2. Questi valori verranno assegnati ai parametri formali `x` e `y`, e la funzione effettuerà lo scambio dei valori.

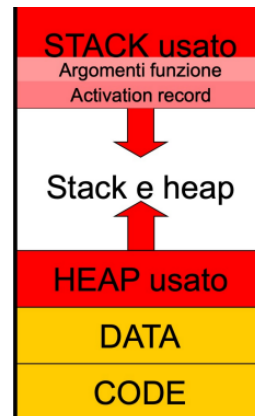
Argomenti e parametri devono corrispondere in base alla posizione e al numero (almeno per le funzioni che definiremo noi). I nomi dei parametri sono indipendenti dai nomi delle variabili del Chiamante. Se la funzione non richiede parametri è preferibile indicare `void` tra le parentesi. In memoria i parametri sono del tutto distinti e indipendenti dagli argomenti, quindi cambiare il valore di un parametro non modifica l'argomento corrispondente.

In merito alla *chiamata di funzione*:

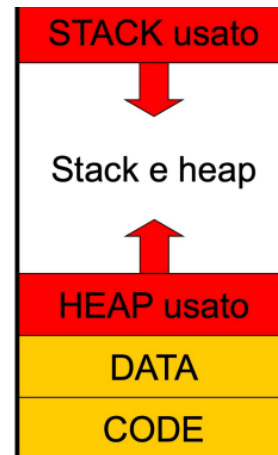
- Il programma compilato è costituito da due parti distinte:
- segment del codice: codice eseguibile
- segmento dei dati: costanti e variabili
- Quando il programma viene eseguito, il Sistema Operativo alloca spazio di memoria per:
- il segmento del codice (CS)
- il segmento dei dati (DS)
- lo stack e lo heap (condivisi)



- Il codice di una funzione è in code
- i parametri e le variabili locali di una funzione vengono allocati nello stack (pila)
- vengono prima copiati i valori dei suoi argomenti e poi vi viene allocato un
- Activation Record (anche detto stack frame) in cui sono allocate le variabili locali della funzione e altro
- Quando la funzione termina, l'AR e gli argomenti vengono rimossi dallo stack che quindi ritorna nello stato



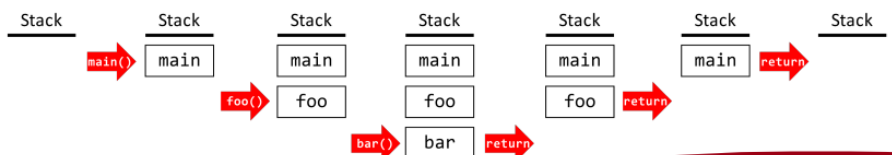
- Nell'Activation Record viene anche memorizzato **l'indirizzo di ritorno dalla funzione**: l'indirizzo di memoria che contiene l'istruzione del modulo chiamante da cui continuare l'esecuzione dopo che la funzione è terminata
- Queste operazioni di allocazione e deallocazione di spazio sullo stack e in generale il meccanismo di chiamata e ritorno da una funzione richiedono tempo



```
void bar() {}
```

```
void foo() { bar(); }
```

```
int main() {
    foo();
}
```



Il calcolatore è in grado di elaborare differenti tipi di informazione e le procedure di codifica/decodifica vengono eseguite dal calcolatore, quindi devono essere pensate in modo che i dati siano facilmente manipolabili dall'elaboratore (più che facilmente comprensibili dall'uomo).

Segue la *rappresentazione di interi positivi*:

- Una sequenza di cifre forma un numero secondo la seguente convenzione: $374 = 3 \cdot 10^2 + 7 \cdot 10^1 + 4 \cdot 10^0$
- Per determinare il valore di un numero binario positivo, si utilizza lo stesso algoritmo dove però la base è 2:
 $(1101)_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 1 = 13$
- Il numero di configurazioni diverse di n bit è 2^n , per cui si riescono a rappresentare 2^n numeri diversi.
- Il numero più grande rappresentabile con n bit è $2^n - 1$ (perché si inizia a contare da 0). Il calcolatore non può rappresentare infiniti numeri!

- Trasformazione da base 10 a base k
- 1. Dividere il numero per k
- 2. tenere traccia del resto
- 3. se il quoziente è maggiore di 0 ripetere il passo 1 con il quoziente
- 4. scrivere i resti nell'ordine inverso rispetto al quale sono stati ottenuti

numero	quoziente	resto
43/2	21	1
21/2	10	1
10/2	5	0
5/2	2	1
2/2	1	0
1/2	0	1

$$(43)_{10} = (101011)_2$$

Per quanto riguarda la *rappresentazione di interi con segno*:

- Il bit più a sinistra rappresenta il segno: 0 = positivo, 1 = negativo
- se si utilizzano n bit, si riescono a rappresentare tutti i numeri
 $x. - (2^{n-1} - 1) \leq x \leq 2^{n-1} - 1 \Leftrightarrow -2^{n-1} + 1 \leq x \leq 2^{n-1} - 1$

Es. con 4 bit si rappresenta [-7,7]

Binario	Decimale	Binario	Decimale
0000	0	1000	0
0001	1	1001	-1
0010	2	1010	-2
0011	3	1011	-3
0100	4	1100	-4
0101	5	1101	-5
0110	6	1110	-6
0111	7	1111	-7

Segue la rappresentazione dei numeri *in complemento a 2*:

- I numeri positivi sono rappresentati in modo "standard" (come nella notazione modulo e segno), utilizzando n bit
- I numeri negativi sono rappresentati "in complemento a 2", ovvero si somma 2^n al numero e poi rappresenta in modo "standard". Es. n = 4

$$(-3)_{10} \rightarrow 2^4 - 3 = 16 - 3 = (13)_{10} = (1101)$$

Rappresentazione in complemento a 2 con 4 bit

Binario	Decimale	Binario	Decimale
0000	0	1000	-8
0001	1	1001	-7
0010	2	1010	-6
0011	3	1011	-5
0100	4	1100	-4
0101	5	1101	-3
0110	6	1110	-2
0111	7	1111	-1

Una sola rappresentazione per lo zero

I numeri interi positivi sono rappresentati all'interno dell'elaboratore utilizzando un multiplo del byte (generalmente 4 o 8 byte).

Se si vuole verificare la dimensione di un int, il comando `sizeof(int)` restituisce il numero di byte (celle di memoria) occupati da un int. Il file `limits.h` (`#include`) riporta una serie di costanti numeriche, tra cui il massimo numero rappresentabile sul computer che si sta usando: `INT_MAX = 2147483647`.

`INT_MAX + 1 = -2147483648` //dovuto alla rappresentazione in complemento a 2

Non esiste un errore di overflow

Varianti del tipo int

- long x: un intero che usa il doppio dei bit di un int `[printf("%ld",x)]`
- short x: un intero che usa la metà dei bit di un int `[printf("%hd",x)]`
- unsigned int x: un intero positivo `[printf("%u",x)]`
- unsigned long x: un long positivo `[printf("%lu",x)]`

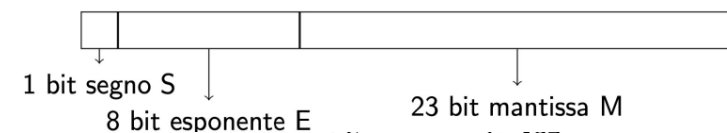
Per quanto riguarda i *numeri reali*, essi utilizzano la rappresentazione in virgola mobile.

Si basa sulla notazione scientifica $1.40 \cdot 10^2 = 140$ (notate che c'è solo una cifra intera, ovvero la notazione è normalizzata)

Lo standard IEEE 754 prevede vari tipi di numeri in virgola mobile, tra cui:

- singola precisione (32 bit) e doppia precisione (64 bit)

i numeri a singola precisione hanno il seguente formato:

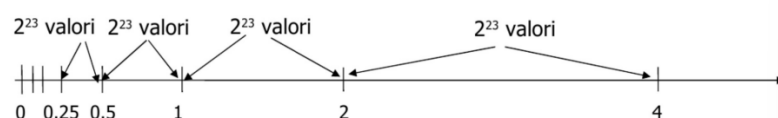


Il formato IEEE 754 è $(-1)^S \cdot 1.M \cdot 2^{E-127}$

1.M, la mantissa indica il numero vero e proprio in forma normalizzata

2^{E-127} indica dove mettere la virgola (moltiplicare/dividere per 2 un numero binario significa spostare a destra/sinistra la virgola di una posizione)

- Intervallo rappresentabile in singola precisione: circa da $1.4 \cdot 10^{-45}$ a $6.81 \cdot 10^{38}$
- in totale si riescono a rappresentare 2^{32} numeri distinti (metà positivi, metà negativi)



- I numeri rappresentabili non sono distribuiti uniformemente, ma sono più densi vicino allo zero.

In merito ai *caratteri*, Lo standard di codifica più diffuso è il codice ASCII, per American Standard Code for Information Interchange. Esso definisce una tabella di corrispondenza fra ciascun simbolo (carattere minuscolo, maiuscolo, cifre) e un codice a 7 bit (128 caratteri).

Vi è poi UNICODE (UTF-8 e UTF-16): standard proposto a 8 e 16 bit (65.536 caratteri).

Il tipo carattere viene definito con *char* e normalmente si assegna un carattere con degli apici.

- Esempio: *char x='c';*

Tabella ASCII

	0	1	2	3	4	5	6	7
0	NUL	DLE	space	□	@	P	`	p
1	SOH	DC1 XON	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3 XOFF	#	3	C	S	c	s
4	EOT	DC4	\$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	.	L	Z	j	z

Gli operatori aritmetici ed i comandi sono definiti tra termini dello stesso tipo • Ma il C effettua automaticamente alcune conversioni tra tipi, le *promozioni*:

- il tipo con meno capacità espressiva viene promosso al tipo con maggiore capacità espressiva (ovvero presi due elementi nella tabella a fianco, la conversione avviene a quello più in alto)
- quando si cambia il tipo di un'espressione (non di una variabile), viene fatta una copia del valore temporanea per effettuare il calcolo

- Attenzione: alcune conversioni, per esempio tra int e unsigned int, possono produrre effetti inaspettati.

```
int g = -6;
unsigned int gg = 3;
printf("%u\n", gg+g);
printf("%d\n", gg+g);
```

• Stampa:
4294967293
-3

- E' possibile forzare la conversione ad un tipo. Es.

```
int x = (int) 3.2 + 3;
printf("%d", x); // stampa 6!
```

- Attenzione che forzando la conversione, per esempio da float a int, si può perdere informazione!

Si rischia di perdere informazione, ovvero il C non è type-safe.

Array

Un array è un gruppo di locazioni di memoria contigue che hanno tutte lo stesso tipo.

Dichiarazione: tipo nome[dimensione];

Es. `int c[12];` // dichiara un array (una sequenza) di 12 interi

`c[i]` si comporta come una variabile

`c[i]` accede all'i-esimo elemento dell'array (si usa la parola indice per riferirsi al numero tra `[]`):

- Il primo elemento ha indice 0
- l'ultimo ha indice dimensione-1

Es. `printf("secondo elemento di c=%d", c[1])`

`c[2] = 1;` // il valore del terzo elemento dell'array passa da 0 a 1

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Ad esempio:

```
int n[5];
for (int i = 0; i < 5; i=i+1) { // inizializza a zero gli elementi dell'array
    n[i] = 0;
}
for (int i = 0; i < 5; i=i+1) { // stampa gli elementi dell'array
    printf("%d = %d\n", i, n[i]);
}
int n[5] = {32, 27, 64, 18, 95}; // dichiara ed inizializza l'array
int n[] = {32, 27, 64, 18, 95}; // dichiarazione equivalente
```

Il C non ha meccanismi di controllo dei confini di un array. Un programma può fare riferimento a un elemento che non esiste e non ricevere un errore!

- `int n[5] = {32, 27, 64, 18, 95};`
- `for (int i = 0; i < 6; i=i+1) { // stampa gli elementi dell'array`
`printf("%d = %d\n", i, n[i]);`
`}`
- Nell'ultima iterazione si stampa il contenuto della cella di memoria seguente all'array; se siamo fortunati questo genera un errore, altrimenti viene stampato un valore imprevedibile.

Una sequenza di caratteri viene anche chiamata stringa.

- `char string1[] = "ciao mondo";`
`printf("la stringa inizia per %c\n", string1[0]); // stampa c`
`printf("%s\n", string1); // stampa ciao mondo, %s sta per stringa`

Ora definiamo nel dettaglio le due funzioni citate per acquisizione input e stampa, parte della libreria standard *stdio.h*:

- `scanf` è utilizzato per leggere dati da un flusso di input, come il tastierino o un file. La sintassi di base di `scanf` è la seguente:
 - `scanf("%formato", &variabile);`
- dove `%formato` è una stringa di formato che specifica il tipo di dati che verranno letti (ad esempio, `%d` per un intero, `%f` per un numero a virgola mobile, `%s` per una stringa), e `&variabile` è un puntatore alla variabile in cui verranno memorizzati i dati letti.
- `printf` è utilizzato per scrivere dati su un flusso di output, come lo schermo o un file. La sintassi di base di `printf` è la seguente:
 - `printf("formato", variabile);`
- dove `formato` è una stringa di formato che specifica come verranno stampati i dati (ad esempio, `%d` per un intero, `%f` per un numero a virgola mobile, `%s` per una stringa), e `variabile` è la variabile che contiene i dati da stampare.

Entrambe le funzioni restituiscono un valore intero che indica il numero di elementi elaborati correttamente. Questo può essere utilizzato per verificare se l'elaborazione è stata completata correttamente o se ci sono stati errori.

Puntatori

18

I puntatori sono uno strumento di basso livello: ci permettono di manipolare altre variabili (altre celle di memoria). Gli indirizzi di memoria (l-valori) sono interi positivi. Un puntatore è una variabile il cui r-valore è un l-valore.

- Es. `int x = 3;` // l-valore = 1021, r-valore = 3 la variabile con l-valore 1025 potrebbe essere un puntatore (vedi figura)

Dichiarazione: `tipo *nome;`

- Es. `int *ptr = NULL;` // l-valore = 1026
- `ptr` è una variabile di tipo puntatore ad una variabile di tipo intero.

Per ogni Tipo di variabile, esiste il corrispondente tipo "puntatore a Tipo"

- `float *x;` `char *c;` `long *l;` ...
- Esistono pure puntatori a funzioni
- `&x` (dove `x` è necessariamente una variabile) restituisce l'l-valore di `x`
- `*p` restituisce la variabile indicata dal r-valore del puntatore `p`
- `&` e `*`
 - o hanno precedenza minore di `() []`, ma maggiore degli operatori aritmetici
 - o sono associativi da destra a sinistra
- `int *xptr = &x;` // l-valore di `xptr` = 1025
- `printf("%p,%d,%p", xptr, *xptr, &x);` // stampa 1021, 3, 1021

1020	
1021	3
1022	
1023	
1024	
1025	1021
1026	0
1027	

1020	
1021	3
1022	
1023	
1024	
1025	1021
1026	0
1027	

```
int x = 3;
```

```
int *xp; // l-valore=1025; r-valore=indefinito
```

```
xp = &x; // r-valore=1021
```

```
*xp = 4; printf("%d",x); // stampa 4
```

```
*xp = *xp+1; printf("%d",x); // stampa 5
```

```
int *xp2 = &x; //ok, ad xp2 si assegna un l-valore
//equivalente a int *xp2; xp2 = &x;
// NON a int *xp2; *xp2 = &x;
```

```
int *xp3 = x; // errore di tipo
```

1020	
1021	3
1022	
1023	
1024	
1025	1021
1026	1021
1027	

6

```
int x = 3; int *xp2;
```

```
*xp2 = x; // xp2 è stata dichiarata ma non inizializzata, per cui l' r-valore
di xp2, *xp2, punta ad una cella di memoria con un valore imprevedibile nella
quale cerchiamo di scrivere un intero, perciò ci aspettiamo (e speriamo in) un
errore di Segmentation Fault.
```

- Come evitare il problema?

Attenzione:

- Non usare una variabile non inizializzata (occorre controllare che il puntatore non sia nullo)

```
int *xp2 = NULL; //equivalente a int *xp2 = 0, l-valore=1026
```

```
if (!xp2) {
```

```
    // puntatore non inizializzato
```

```
}
```

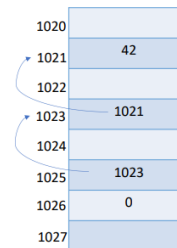
- Controllare che il tipo della variabile sia corretto per l'operazione dove si sta per usare.
- `&*` sono associativi da destra `&*p = &(*p)`.
- Memorizzare alla perfezione il significato degli operatori: per esempio `*` restituisce la variabile (un oggetto che ha un nome, tipo, l-valore, r-valore) non il suo r-valore.
- `*` è usato con due significati diversi 1) nella dichiarazione e 2) nel corpo di una funzione.

Se un puntatore è stato dichiarato ed inizializzato, può essere utilizzato in un'espressione.

```
int y, x=3, *p1=&x, *p2=p1;
y = *p1 * *p2; // (*p1) * (*p2)
y = x + *p2;
y = 5* - *p2/ *p1; // se non ci fosse stato lo spazio tra / e *?
```

Si può definire un puntatore ad una variabile puntatore utilizzando più volte il simbolo `*` nella dichiarazione
`int x, *p1;`

```
int **p2; // p2 è un puntatore ad una variabile
           //puntatore a variabile intera. L-valore: 1025
x = 42; p1 = &x;
p2 = &p1;
printf("%d", **p2);
```



Distinguiamo i passaggi di parametri in due tipi: *per valore* e *per riferimento*.

- **Passaggio per valore:** i valori dei parametri attuali vengono assegnati ai parametri formali al momento dell'invocazione della funzione.
 - La funzione (f) non modifica le variabili parametri attuali (y)

```
void f(int x) {
    x = x + 1;
}

int y = 2;
f(y);
printf("%d", x); // stampa 2
```

Il passaggio per riferimento permette di modificare all'interno della funzione i parametri attuali (y). In C è implementato solamente il passaggio per valore. Il passaggio per riferimento è ottenuto passando per valore il puntatore ad una funzione.

```
void f(int x) {
    x = x + 1;
}

int y = 2;
f(y)
printf("%d", x); // stampa 2
```

Passaggio per Riferimento: Esempi

- Scambio di variabili

```
void scambio(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int x=2, y=5;
    scambio(&x, &y);
    printf("x=%d, y=%d\n", x,y); //stampa x=5, y=2
}
```

Notiamo inoltre le seguenti cose:

- `int x[] = {1,2,3,4,5}`
 - `x` è un puntatore **costante** al primo elemento dell'array:
 - `x == &x[0]`
 - `int *p = x;` // corretto, equivale a `&x[0]`
//assumendo che l-valore di `x` sia 1022
- `&x[0] == p == 1022`
`&x[1] == p+1 == 1022 + 1*sizeof(int) == 1026`
`&x[2] == p+2 == 1022 + 2*sizeof(int) == 1030`
`&x[3] == p+3 == 1022 + 3*sizeof(int) == 1034`

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Riferendoci, invece, all'*aritmetica dei puntatori*:

Si può sommare o sottrarre un intero ad un puntatore

- `int x, *p = &x; p+2` equivale a `p+2*sizeof(int)`

Si può sottrarre due puntatori (ha senso se si riferiscono allo stesso array)

- `p1-p2`

Si possono confrontare due puntatori:

- `p1>p2`, `p1==p2`, `p1!=p2`

Non è ammesso utilizzare puntatori in moltiplicazioni o divisioni

- `p1/3`, `p1/p2`, `p1*p2`

Non è ammesso sommare due puntatori

- `p1+p2`

Esempio: Stampa Array



```
int a[] = {1,2,3,4};
int i = 0, *p = a;
while(i<4) {
    printf("%d\n", *p);
    i+=1;
    p = p+1;
}
```

Stampa:

```
1
2
3
4
```

Esempio: Stampa Array (versioni 2 - 3)

```

int a[] = {1,2,3,4};
int i = 0, *p = a;
while(i<4) {
    printf("%d\n", *(p+i));
    i+=1;
}
// Oppure
while(i<4) {
    printf("%d\n", *(a+i));
    i+=1;
} //ma a+=i sarebbe sbagliato!

```

Entrambi Stampano:

```

1
2
3
4

```

17

Array Come Parametri di Funzione

- Quando passiamo un array ad una funzione, in realtà passiamo il puntatore al primo elemento dell'array
- In C gli array sono sempre passati per riferimento

```

void successivo(int array[], int N) {
    for(int i=0; i<N; i+=1) {
        array[i] += 1;
    }
}

```

```

int main (void) {
    int a[]={1,2,3};
    successivo(a,3);
    //a={2,3,4}
}

```

Un esempio visivo di passaggio di parametri con la classica funzione di scambio:

```

void scambio(int *x, int *y) { // int *x = &w; int *y = &z
    int temp = *x;
    *x = *y;
    *y = temp;
}
→ int main() {
    int w=2, z=5;
    scambio(&w, &z);
    printf("w=%d, z=%d\n", w,z); //stampa w=5, z=2
}

```

1020	
1024	
1040	
1044	
1048	
1052	

```

void scambio(int *x, int *y) { // int *x = &w; int *y = &z
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main() {
    → int w=2, z=5;
    scambio(&w, &z);
    printf("w=%d, z=%d\n", w,z); //stampa w=5, z=2
}

```

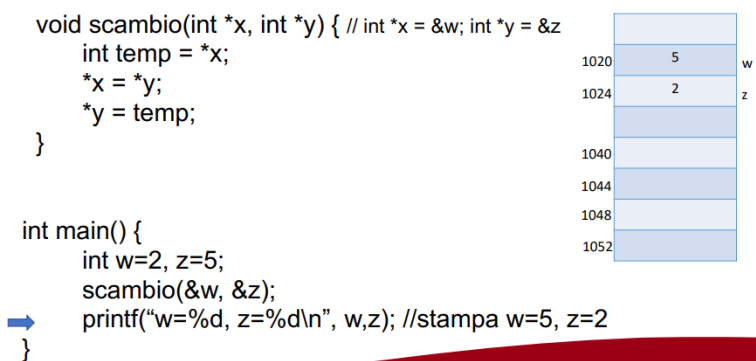
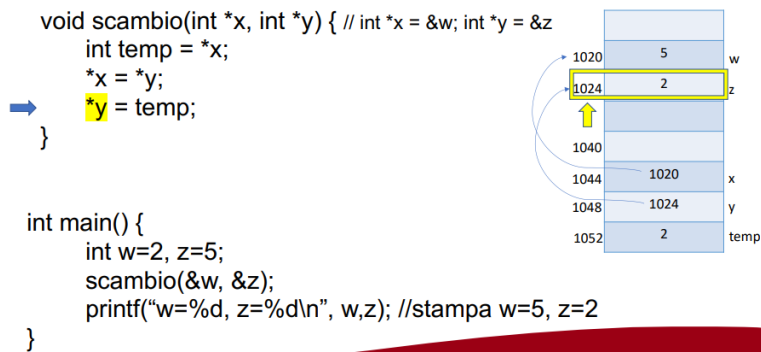
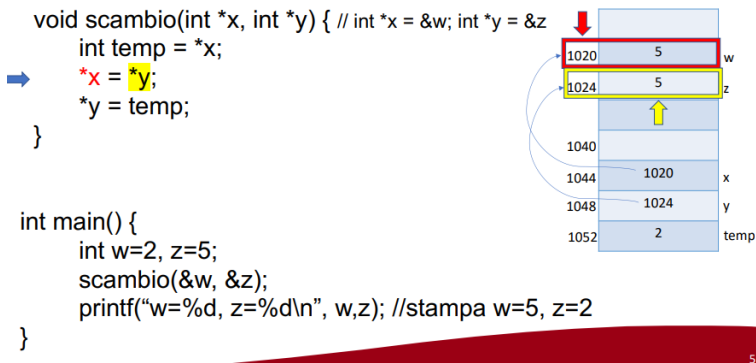
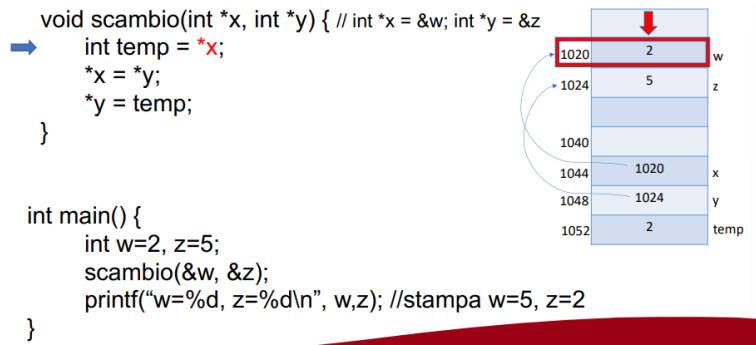
1020	2	w
1024	5	z
1040		
1044		
1048		
1052		

```

→ void scambio(int *x, int *y) { // int *x = &w; int *y = &z
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main() {
    int w=2, z=5;
    scambio(&w, &z);
    printf("w=%d, z=%d\n", w,z); //stampa w=5, z=2
}

```

1020	2	w
1024	5	z
1040		
1044	1020	x
1048	1024	y
1052		



Una funzione può restituire un puntatore. Bisogna stare però attenti: le variabili create all'interno della funzione vengono deallocate (distrutte) al termine dell'esecuzione. Qui si ha a che fare con una *dangling reference*, cioè un collegamento o un puntatore a un'istruzione, a un elemento di tabella, a un elemento di indice e così via che non contiene più lo stesso contenuto. Se il riferimento non è un indirizzo attualmente valido, o se è valido ma non c'è contenuto in quella posizione, può causare il blocco del computer se il software non è programmato con attenzione.

- anche min viene deallocata (il contenuto della cella di memoria potrebbe essere utilizzato per creare altre variabili)

```
int a[] = {1,2,3,4}
p = min(a, 4);
printf("\n%d\n", *p);
```

```
int * min(int*X, int dimX) {
    int i=1, min=X[0];
    while(i < dimX) {
        if(X[i] < min)
            min=X[i];
        i=i+1;
    }
    return &min;
}
```

Tornando a parlare di stringhe,

consideriamo `char string1[] = "ciao";`

In memoria viene memorizzato come `"ciao\0"`, per cui serve un array con `n+1` elementi, dove `n` è la lunghezza della stringa (5 nell'esempio)

- il carattere `\0` indica la fine della stringa
- il primo elemento di `string1` è un `char`

1020	
1021	c
1022	i
1023	a
1024	o
1025	\0
1026	
1027	

La prima fase della traduzione da codice C a linguaggio macchine è eseguita dal preprocessore che elimina i commenti e

- crea costanti simboliche che sono *direttive al preprocessore*
- *#define identificatore testo_di_sostituzione*

- c'è solo uno spazio tra l'identificatore e il testo sostituito (non c'è ; dopo!)

```
#define PI 3.1415
```

```
int main(void) {
    float x = 2*PI;
}
```

Preprocessore

```
int main(void) {
    float x = 2*3.1415;
}
```

In merito agli *operatori logici*:

Operatore	Significato	Esempio
&& (binario)	AND logico. Vero solo se entrambi gli operandi sono Veri	((5==5) && (5>2)) è Falso.
(binario)	OR logico. Vero se almeno uno degli argomenti è Vero	((5==5) (5>2)) è Vero

&& e || sono valutati in modo lazy (pigro).

- Se A e B sono due condizioni per `(A && B)` e `(A || B)` si valuta sempre prima A:
- se A è falsa, `A && B` restituisce Falso senza valutare B
- se A è vera, `A || B` restituisce Vero senza valutare B

Gli array possono avere più di una dimensione (es. per rappresentare tabelle, matrici, od oggetti multidimensionali).

- tipo `nome[indice1][indice2]...`

- Es. `int x[3][4];`

	Colonna 0	Colonna 1	Colonna 2	Colonna 3
Riga 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Riga 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Riga 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Indice di colonna
Indice di riga
Nome dell'array

• Inizializzazione:

• `int ar[2][3] = { {1, 2, 3}, {4, 5, 6} };`

es. `a[1][0]` `{0 1 2}`

• Anche gli array multidimensionali sono salvati come una sequenza contigua di variabili, riga per riga. Le seguenti inizializzazioni sono equivalenti:

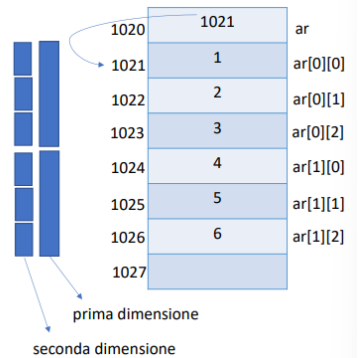
• `int ar[2][3] = {1, 2, 3, 4, 5, 6};`

• `int ar[][3] = {1, 2, 3, 4, 5, 6};` // solo la prima dimensione può essere lasciata in bianco

• `int ar[][3] = {1, 2, 3, 4, 5, 6};` // solo la prima dimensione può essere lasciata in bianco

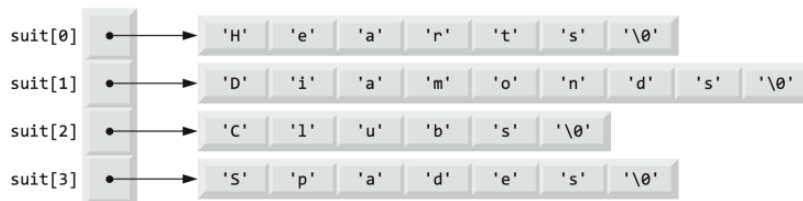
• Perché le altre devono esserci? Se accedo a `ar[1][0]` devo sapere che devo saltare 3 elementi (`ar` è solo un puntatore al primo elemento dell'array)!

• Gli stessi ragionamenti si applicano ad array di più dimensioni: `int *X[2][3][4];`



I puntatori sono più generali, permettono di avere righe di dimensione diversa:

`char *suit[4] = {"Hearts", "Diamonds", "Clubs", "Spades"};`



Una stringa può essere rappresentata da

• un array di caratteri, se si aggiunge '\0' alla fine.

• La dichiarazione riserva un certo numero di celle di memoria. Quindi Si riesce a modificare un singolo carattere (`s[0]='K'`)

• un puntatore a char, a cui può essere assegnata una stringa costante, della quale non si possono modificare i caratteri. Si può riassegnare una seconda stringa al puntatore.

```
char s[8] = "Hearts";
char *ps = "Hearts";
printf("%s - %s\n\n", s, ps);
s[0] = 'K';
ps = "Ke";
printf("%s - %s\n", s, ps);
--Output--
Hearts - Hearts
Kearts - Ke
```

Correttezza: PRE e POST Condizioni

Fino ad ora abbiamo detto che un programma è corretto se realizza la consegna. Cerchiamo di essere più formali. Anche se si può dimostrare la correttezza di frammenti di codice o interi programmi, generalmente si dimostra la correttezza di una funzione.

- La Precondizione (PRE) indica ciò che si assume vero prima della chiamata della funzione
 - o Siamo liberi di scegliere la PRE come vogliamo, ma dovrebbe essere più generale possibile, affinché la nostra funzione sia utilizzabile in più scenari possibili.
 - o Chi usa la funzione deve rispettare la PRE; chi la crea la assume vera
- La Postcondizione (POST) indica ciò che sarà vero dopo che la funzione sarà eseguita. La POST deve essere utile per dimostrare la consegna

```
int radice_quadrata (float x) {
    //PRE: x>=0
    ....
    //POST: restituisce la radice quadrata di x
}
```

- La PRE e la POST sono le informazioni che servono a chi deve usare la funzione
 - La POST ci dice cosa calcola
 - La PRE, assieme al prototipo, come dobbiamo invocarla
 - Per chi usa la funzione, l'implementazione è irrelevante se la POST è dimostrata

Se il programma non è corretto, basta fornire un esempio di input/output che non sia corretto.

```
int radice_quadrata (float x) {
    //PRE: x==9
    return 3;
    //POST: restituisce la radice quadrata di x
} // corretta ma non interessante, la funzione può essere applicata solo al numero 9!
```

```
int radice_quadrata (float x) {
    //PRE:
    if (x<0) {return -1;}
    ....
    //POST: se x<0 restituisce -1, altrimenti restituisce la radice quadrata di x
}
```

Al posto dell'if possiamo usare la funzione *assert*, che genera un errore se la condizione al suo interno è falsa e termina il programma:

```
#include <assert.h>
int radice_quadrata (float x) {
    //PRE: x>=0
    assert(x>=0);
    ....
    //POST: restituisce la radice quadrata di x
}
```

```
void minimo(int x, int y, int z) {
    /* PRE */
    if ( ( x <= y ) && ( x <= z ) )
        printf("%d", x);

    if ( ( y <= x ) && ( y <= z ) )
        printf("%d", y);

    if ( ( z <= x ) && ( z <= y ) )
        printf("%d", z);
    /* POST stampa un numero che è il valore minimo tra le 3 variabili
```

Abbiamo inoltre l'invariante di un ciclo è una proprietà che è vera • prima, • durante e • dopo un ciclo. Si usa per dimostrare la POST quando il codice ha un ciclo.

C'è una relazione diretta tra l'invariante e la post:

- all' iterazione i-esima del ciclo si è risolto il problema fino all'i-esimo elemento della lista (invariante)
- la POST si osserva applicando l'invariante a tutta la lista

```

i=0
// invariante vera qua
while (i<N) {
    // invariante vera qua
    printf("%d\n", i)
}
// invariante vera qua

int somma(int *X, int size) {
    //PRE l'array X ha dimensione size
    int sum = 0;
    for (int i=0; i<size; i=i+1) {
        //INV sum=0+X[0]+...+X[i-1]
        sum += X[i];
    }
    return sum;
    //POST i=size. sum=X[0]+...+X[size-1] (restituisce la somma dei valori dell'array)
}

```

```

int minimo(int *X, int size) {
    //PRE l'array X ha dimensione size
    int min = X[0];
    for (int i=1; i<size; i=i+1) {
        //INV per ogni 0<=j<i. min<=X[j] (min è il minimo tra X[0] e X[i])
        if(X[i]<min)
            min = X[i];
    }
    return min;
    //POST per ogni 0<=j<size. min<=X[j] (min è il minimo dell'array)
}

```

* Implementare un'applicazione per cifrare/decifrare messaggi.

*

* - Cifratura -

*

* La cifratura avviene modificando solamente i caratteri

* alfanumerici (ovvero le cifre 0-9, le lettere minuscole

* a-z, le lettere maiuscole A-Z) del messaggio nel

* modo seguente: dato il valore numerico

* corrispondente ad un carattere (es. a=97, A=65), e dato un

* intero k!=0, la cifratura di un carattere avviene sommando k

* al valore numerico del carattere.

* Ad esempio se k=2 'a' viene trasformata in 'c', 'B' in 'D', '1'

* in '3', '!' in '!' (perché non è un carattere alfanumerico).

* Inoltre si deve assicurare che ogni cifra

* sia trasformata in una cifra, ogni lettera minuscola in

* una lettera minuscola, ogni lettera maiuscola in una

* lettera maiuscola. Ciò si ottiene considerando 'a' il

* carattere seguente di 'z', 'A' quello di 'Z', '0' quello di '9'.

* Ad esempio se k=3 'y' viene trasformata in 'b', 'Z' in 'C',

* '7' in '0'

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#define MAX_LEN 100
```

```
void encrypt(char message[], int k) {
```

Scritto da Gabriel

```

for (int i = 0; message[i]; i++) {
    if (isdigit(message[i])) {
        message[i] = (message[i] - '0' + k) % 10 + '0';
    } else if (islower(message[i])) {
        message[i] = (message[i] - 'a' + k) % 26 + 'a';
    } else if (isupper(message[i])) {
        message[i] = (message[i] - 'A' + k) % 26 + 'A';
    }
}

void decrypt(char message[], int k) {
    for (int i = 0; message[i]; i++) {
        if (isdigit(message[i])) {
            message[i] = (message[i] - '0' - k + 10) % 10 + '0';
        } else if (islower(message[i])) {
            message[i] = (message[i] - 'a' - k + 26) % 26 + 'a';
        } else if (isupper(message[i])) {
            message[i] = (message[i] - 'A' - k + 26) % 26 + 'A';
        }
    }
}

int main() {
    char message[MAX_LEN];
    int k;
    printf("Enter the message: ");
    scanf("%s", message);
    printf("Enter the key: ");
    scanf("%d", &k);
    encrypt(message, k);
    printf("Encrypted message: %s\n", message);
    decrypt(message, k);
    printf("Decrypted message: %s\n", message);
    return 0;
}

```

In C89 la dimensione di un array deve essere una costante intera positiva (no const), nota a tempo di compilazione (non una variabile, letta da tastiera o meno). Di norma la costante si indica tramite #define

- #define DIM_ARRAY 3
- int x[DIM_ARRAY] = {1,2,3} //OK

Dal C99 è possibile dichiarare array la cui dimensione non è fissata a tempo di compilazione • Tali array sono implementati internamente in modo diverso dal C. Una delle differenze è che non possono essere inizializzati • int n = 3; int x[n] = {1,2,3}; //NO • x[0]=1; x[1]=2; x[2]=3; //OK

Ricorsione: tipi di ricorsione, correttezza ed induzione

Una funzione è detta ricorsiva se chiama sé stessa.

- Se due funzioni si chiamano l'un l'altra, sono dette mutuamente ricorsive
- La funzione ricorsiva sa risolvere direttamente solo casi particolari di un problema detti casi di base:
 - o se viene invocata passandole dei dati che costituiscono uno dei casi di base, allora restituisce un risultato
 - o se invece viene chiamata passandole dei dati che NON costituiscono uno dei casi di base, allora chiama sé stessa (passo ricorsivo) passando dei DATI semplificati/ridotti

Un esempio di funzione ricorsiva è il calcolo del fattoriale, che normalmente si indica con $n!$ ed è uguale al prodotto di tutti i numeri interi positivi minori o uguali a n .

$Fatt(0) = 1$ (caso base)

$Fatt(n) = n * Fatt(n-1)$ (caso ricorsivo)

$Fatt(3) = 3 * Fatt(2)$

$Fatt(2) = 2 * Fatt(1)$

$Fatt(1) = 1 * Fatt(0)$

$Fatt(0) = 1$

quindi

$Fatt(n-1) = (n-1) * Fatt(n-2)$ quindi

$Fatt(n) = n * (n-1) * \dots * 1$

$Fatt(1) = 1 * 1$

$Fatt(2) = 2 * (1 * 1)$

$Fatt(3) = 3 * (2 * 1)$

Spesso la ricorsione permette di risolvere un problema anche molto complesso con poche linee di codice.

La ricorsione è poco efficiente perché richiama molte volte una funzione e questo:

- richiede tempo per la gestione dello stack
- consuma molta memoria (alloca un nuovo stack frame a ogni chiamata, definendo una nuova ulteriore istanza delle variabili locali e dei parametri ogni volta)

Qualsiasi problema ricorsivo può essere risolto in modo non ricorsivo (ossia iterativo), ma la soluzione iterativa potrebbe non essere facile da individuare oppure essere (molto) più complessa

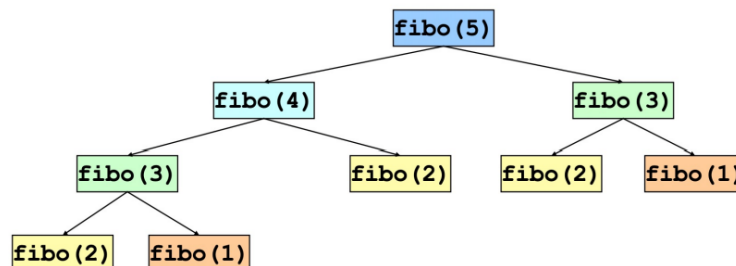
L'approccio ricorsivo è in genere da preferire se:

- non ci sono particolari problemi di efficienza e/o di memoria
- è più intuitivo di quello iterativo
- la soluzione iterativa non è evidente o agevole

Ricorsione: Esempio

- La nostra funzione fibo ripete gli stessi calcoli più volte -> inefficiente

```
int fibo(int n) {
    if (n <= 2)
        return 1;
    else
        return fibo(n-1) + fibo(n-2);
}
```



Per le funzioni ricorsive:

- si modella un problema per casi
 - o dal più semplice (caso base)
 - o al più complesso (caso ricorsivo), nel quale si esprime il problema in termini di un caso più semplice
- si esprime la "complessità" mappandola sui numeri naturali
 - o maggiore il numero n , maggiore la complessità
 - o i casi base sono collegati ai valori minori minimi che n può assumere nel nostro problema (casi base)

- $Fattoriale(0) = 1$; $Fattoriale(n) = n * Fattoriale(n-1)$;

Nel caso ricorsivo si esprime il problema per il caso n in funzione di un valore minore di n ; si continua fino a raggiungere i casi base:

1. Identificare la variabile n da cui dipende la complessità del problema
2. Riconoscere tutti i casi base (se ne perdiamo uno, il programma potrebbe non terminare mai, usare Ctrl-c per interrompere l'esecuzione da terminale)
3. Esprimere il problema per il caso n in funzione di un caso $n_1 < n$

Una funzione ricorsiva può essere utilizzata quando la soluzione di un problema può essere espressa in termini della soluzione di un'istanza più semplice dello stesso problema.

Se vogliamo applicare una soluzione ricorsiva dobbiamo:

- chiarire cosa rende un'istanza di un problema più semplice di un'altra. È possibile associare un numero naturale a tale complessità;
- identificare i casi base, ovvero le istanze più semplici del problema, e come calcolare le loro soluzioni (caso base);
- chiarire come, una volta calcolata la soluzione di complessità n , calcolare da essa la soluzione del caso $n+1$ (caso ricorsivo/induttivo).

- Definire una funzione ricorsiva che calcoli il valore minore in un array di interi.

1. Chiarire cosa rende un'istanza di un problema più semplice di un'altra. E' possibile associare un numero naturale a tale complessità

la complessità del problema può essere espressa in termini della dimensione dell'array

- Definire una funzione ricorsiva che calcoli il valore minore in un array di interi.

1. la complessità del problema può essere espressa in termini della dimensione dell'array
2. il caso base è l'array con un solo elemento

Il minimo di un array con un solo elemento è l'unico elemento dell'array

- Definire una funzione ricorsiva che calcoli il valore minore in un array di interi.

1. la complessità del problema può essere espressa in termini della dimensione dell'array
2. il caso base è l'array con un solo elemento
3. Se conoscessi il minore di un array di dimensione n , come potrei trovare la soluzione per un array di dimensione $n+1$, ovvero l'array di dimensione n con un elemento aggiunto?

Se conosco $\min(X[0], \dots, X[n-1])$ come posso calcolare $\min(X[0], \dots, X[n])$?

$\min(X[0], \dots, X[n]) = \min(X[0], \min(X[0], \dots, X[n-1]))$

```

int minimo(int X[], int dim) {
    /*
     * PRE: dim >= 0 è il numero di elementi di X
     */
    if(dim==0)
        return -1; //ERRORE - vettore vuoto
    if(dim==1)
        return X[0];
    else {
        int min_vettore = minimo(X+1, dim-1);
        return (X[0]<min_vettore)? X[0]: min_vettore;
    }
}

```

Per dimostrare la correttezza della ricorsione, si usa il principio di induzione.

$$\frac{P(0) \quad P(n) \Rightarrow P(n+1)}{\forall n. P(n)}$$

- Per dimostrare che una proprietà vale per tutti i numeri naturali, dimostriamo che
- vale per 0
- se assumiamo che vale per n, dimostriamo che vale per n+1

Vediamo l'esempio nel calcolo del valore minore di array:

<pre> int minimo(int X[], int dim) { if(dim==0) return -1; //ERRORE - vettore vuoto if(dim==1) return X[0]; else { int min_vettore = minimo(X+1, dim-1); return (X[0]<min_vettore)? X[0]: min_vettore; } } </pre> Caso base: X ha un elemento, X[0] è il minimo	<pre> int minimo(int X[], int dim) { /* POST restituisce Y=min(X[0],...,X[dim-1]) */ ... else { int min_vettore = minimo(X+1, dim-1); /* min_vettore=min(X[1],...,X[dim-1]) */ return (X[0]<min_vettore)? X[0]: min_vettore; } } </pre> Caso ricorsivo: per induzione assumiamo che la POST sia vera per la chiamata ricorsiva, Y=min(X[1],...,X[dim-1]), e dimostriamo che minimo calcola il minimo di X[0],...,X[dim-1]: basta confrontare Y con X[0] e restituire il minimo dei due.
---	---

- In una prova per induzione:
- si dimostra la POST per il caso base
- assumendo vera la POST per le chiamate ricorsive all'interno della funzione, si dimostra la POST per il caso ricorsivo

$$\frac{P(0) \quad P(n) \Rightarrow P(n+1)}{\forall n. P(n)}$$

```

int lunghezza_stringa(char *p){
    /* PRE: p è un puntatore a stringa
     * POST: restituisce L(p) la lunghezza della stringa
     * puntata da p, ovvero il numero di caratteri escluso \0 */
    if(*p == '\0')
        return 0; //CASO BASE: L(p)=0
    else { int l = lunghezza_stringa(p+1); // L(p+1)=
        return l + 1; //la stringa puntata da p è composta
        da un carattere iniziale più la stringa puntata da p+1, per
        cui L(p) = L(p+1) + 1=l+1, quindi la POST è verificata
    }
}

```

Si possono avere vari tipi di ricorsione; ad esempio, si ha una *ricorsione in coda* (tail recursion) quando ogni chiamata ricorsiva esegue il calcolo di valori intermedi e li passa all'invocazione successiva per un'ulteriore elaborazione. In questo modo è l'ultima chiamata quella che calcola il risultato finale. Questo deve poi solo essere passato indietro al chiamante con una semplice return alla terminazione di ciascuna delle chiamate.

Esempio di Ricorsione in Coda

31

La struttura tipica ha la forma seguente:

```
tipo funzione(tipo x) {
    y = espressione con x;
    return funzione(y);
}
```

```
int confronta_array(int *X, int *Y, int dim) {
    if (dim==0)
        return 1;
    else {
        if (X[0]!=Y[0])
            return 0;
        else {
            return confronta_array(X+1, Y+1, dim-1);
        }
    }
}
```

Nella ricorsione in coda, i dati automatici della chiamata a funzione, salvati sullo stack nei vari stack frame, non servono più alla funzione quando questa torna in esecuzione e alla chiusura vengono solo scartati: di ciascun stack frame serve solo l'indirizzo di ritorno (e il risultato finale).

Una funzione che realizza la ricorsione in coda è facilmente riscrivibile come funzione iterativa con il vantaggio di consumare meno memoria e di essere più veloce (non ci sono chiamate a funzione).

Esempio di trasformazione da Ricorsione a Iterazione:

<pre>tipo F_ric(tipo x) { if (casobase(x)) { istruzioni_casobase; return risultato; } else { istruzioni_nonbase; return F_ric(riduciComplessita(x)); } }</pre>	<pre>tipo F_iter(tipo x) { while (!casobase(x)) { istruzioni_non_base; x = riduci_complessità(x); } istruzioni_casobase; return caso_base; }</pre>
--	--

Esempio di trasformazione



<pre>int confronta_array(int *X, int *Y, int dim) { if (dim==0) return 1; else { if (X[0]!=Y[0]) return 0; else { return confronta_array(X+1, Y+1, dim-1); } } }</pre>	<pre>int confronta_array(int *X, int *Y, int dim) { int i=0; while (i<dim) { if (X[i]!=Y[i]) return 0; i++; } return 1; }</pre>
--	--

Cosa stampa il seguente codice?

```
int x; int y=2; int *p, *q = &y; int **qq = &p; **qq = 3; printf("%d\n", **qq);
```

Risultato: 3

Cosa stampa il seguente codice?

```
int y=2; int **qq = &&y; printf("%d\n", **qq)
```

Risultato: 2

Scrivere un programma che dato un numero in input verifichi con una funzione ricorsiva se è primo (restituire 1) oppure no (restituire 0).

```
#include <stdio.h>

int is_prime(int n, int i) {
    if (i == 1)
        return 1;
    if (n % i == 0)
        return 0;
    return is_prime(n, i - 1);
}

int main() {
    int n;
    printf("Inserisci un numero: ");
    scanf("%d", &n);
    if (is_prime(n, n / 2) == 1)
        printf("%d è primo.\n", n);
    else
        printf("%d non è primo.\n", n);
    return 0;
}
```

Esempio di Dimostrazione per Induzione



```
int potenza(int base, int esp) {
    /*
     PRE: esp >= 0, base != 0
     POST: restituisce base^esp
    */
    if (esp == 0) {
        return 1;
    } else {
        return base * potenza(base, esp - 1);
    }
}
```

Caso base: $1 = \text{base}^0$

Caso ricorsivo: se $\text{esp} > 0$
 potenza(base, esp-1) restituisce
 $\text{base}^{\text{esp}-1}$ (questa prende il nome di
 ipotesi induttiva),
 quindi
 $\text{base} * \text{base}^{\text{esp}-1} = \text{base} * \text{esp} \Rightarrow \text{POST}$

Progettare e implementare una delle seguenti funzioni ricorsive (a seconda dove siete) e dimostrate la sua correttezza

- int trova_elemento(int X[], int dim, int elem) (SIETE A CASA) /* POST restituisce 1 se elem è in X; 0 altrimenti*
- int array_pari(int X[], int dim) (SEDETE ALLA MIA SINISTRA) /* POST restituisce 1 se tutti gli elementi di X sono pari; 0 altrimenti
- int conta_occorrenze(int X[], int dim, int x) (SEDETE ALLA MIA DESTRA) /* POST restituisce il numero di occorrenze di x in X */

```
#include <stdio.h>
```

```
// Trova l'elemento elem nell'array X di dimensione dim
```

```
int trova_elemento(int X[], int dim, int elem) {
    if (dim == 0)
        return 0;
    if (X[dim - 1] == elem)
```

```
    return 1;
    return trova_elemento(X, dim - 1, elem);
}

// Verifica che tutti gli elementi di X siano pari
int array_pari(int X[], int dim) {
    if (dim == 0)
        return 1;
    if (X[dim - 1] % 2 != 0)
        return 0;
    return array_pari(X, dim - 1);
}

// Conta il numero di occorrenze di x in X
int conta_occorrenze(int X[], int dim, int x) {
    if (dim == 0)
        return 0;
    if (X[dim - 1] == x)
        return 1 + conta_occorrenze(X, dim - 1, x);
    return conta_occorrenze(X, dim - 1, x);
}

int main() {
    int X[] = {2, 4, 6, 8};
    int dim = sizeof(X) / sizeof(X[0]);

    printf("Array: ");
    for (int i = 0; i < dim; i++)
        printf("%d ", X[i]);
    printf("\n");

    int elem = 6;
    int result = trova_elemento(X, dim, elem);
    if (result == 1)
        printf("%d è presente nell'array.\n", elem);
    else
        printf("%d non è presente nell'array.\n", elem);

    result = array_pari(X, dim);
    if (result == 1)
        printf("Tutti gli elementi dell'array sono pari.\n");
    else
        printf("Non tutti gli elementi dell'array sono pari.\n");

    elem = 8;
    result = conta_occorrenze(X, dim, elem);
    printf("%d compare %d volte nell'array.\n", elem, result);

    return 0;
}
```

}

Per dimostrare la correttezza delle tre funzioni, possiamo utilizzare le pre-e post-condizioni.

- La pre-condizione di `trova_elemento` è che l'array `X` sia di dimensione `dim`, e che l'elemento `elem` sia di un tipo compatibile con gli elementi di `X`.
- La post-condizione di `trova_elemento` è che, se `elem` è presente in `X`, la funzione restituisce 1; altrimenti, restituisce 0.
- La pre-condizione di `array_pari` è che l'array `X` sia di dimensione `dim`.
- La post-condizione di `array_pari` è che, se tutti gli elementi di `X` sono pari, la funzione restituisce 1; altrimenti, restituisce 0.
- La pre-condizione di `conta_occorrenze` è che l'array `X` sia di dimensione `dim`, e che `x` sia di un tipo compatibile con gli elementi di `X`.
- La post-condizione di `conta_occorrenze` è che la funzione restituisce il numero di occorrenze di `x` in `X`.

File multipli, inclusione, variabili globali, variabili statiche ed esercizi

Quando i nostri programmi diventano di grandi dimensioni, o per riutilizzare agevolmente funzioni già realizzate, è possibile implementare queste ultime in un file separato, che poi andremo a “collegare” al nostro file principale.

In realtà si creano due file

- `.c` con l'implementazione delle funzioni
- `.h` con le intestazioni (i prototipi) delle funzioni
- Abbiamo già visto alcuni esempi: `stdio.h`, `assert.h`

Nel programma principale, per poter utilizzare le funzioni aggiuntive, basta utilizzare la direttiva `#include`

- `#include (<> fanno sì che si cerca tra i file forniti dal sistema operativo)`
- `#include "stringhe.h" ("" cerca stringhe.h prima nella cartella corrente)`

Se il file `header.h` contiene

- `char *test (void);`
- E nel nostro file principale

```
#include "header.h"
```

```
int main (void) {
    printf("%s\n", test);
}
```

- Allora il preprocessore trasformerà il file principale in

```
char *test (void);
int main (void) {
    printf("%s\n", test);
}
```

Notare che non abbiamo ancora a disposizione l'implementazione di `test`. Questo è accettabile nella fase di compilazione, ma l'implementazione della funzione `test` deve essere raggiungibile nella fase finale di linking. Questo permette la compilazione separata (ma non l'esecuzione!) dei vari file che costituiscono un progetto.

Includere un file header più volte corrisponde ad un errore di compilazione (si dichiara due volte la stessa funzione). Il C mette a disposizione delle direttive del preprocessore per evitarlo.

```
#ifndef HEADER_FILE
#define HEADER_FILE
//contenuto del file header.h: prototipi di funzioni e #define
#endif
```

- le istruzioni tra ifndef ed endif vengono copiate solo se la variabile HEADER_FILE non è definita

In pratica:

- Se si vuole creare un file con una serie di funzioni, per esempio che operano su stringhe, si deve
 - creare il file .h con i prototipi delle funzioni e le #define (proteggere il file dall'inclusione multipla)
 - creare un file .c (con lo stesso nome) che includa il file .h (così il compilatore controlla che le dichiarazioni di funzione sono coerenti)
 - Nella fase di compilazione è sufficiente elencare i file .c che vogliamo compilare (non è necessario includere i .h)
 - gcc -o programma main.c stringhe.c
 - Se i file da compilare sono molti, conviene utilizzare l'utility make per sveltire il processo di compilazione (richiede un file di configurazione, Makefile, nel quale in pratica si specifica il comando gcc)

Le *variabili globali* sono definite fuori da ogni funzione sono dette globali, sono visibili in tutto il file da dove sono definite in poi (a meno che non si definisca una variabile con lo stesso nome in una funzione – in questo caso la variabile globale non è visibile nella funzione).

Le variabili globali sono da in generale da evitare (ogni funzione dovrebbe modificare solamente variabili locali e parametri). Si usano nel caso una variabile debba essere usata in molte funzioni e si debba mantenerne il valore tra ogni chiamata.

```
#include <stdio.h>
```

```
int x=10;
```

```
int main(void) {
    //int x = 8; //questa dichiarazione rende invisibile la variabile globale nel main
    printf("%d\n", x); //se tolgo il commento all'istruzione precedente stampa 8
}
```

Se vogliamo utilizzare una variabile globale definita in un altro file, dobbiamo ridichiarare la variabile con la parola chiave *extern* davanti alla dichiarazione:

- Se non esiste la variabile counter in file1.h non si ottiene un errore compilando main.c (diventa una variabile diversa)

file1.h	main.c
<pre>#ifndef FILE1_H #define FILE1_H int counter; #endif</pre>	<pre>#include "file1.h" extern int counter; int main(void) { counter = 5; printf("%d\n", counter); }</pre>

Parliamo ora di *variabili statiche*, le quali:

- sono create all'inizio dell'esecuzione
- mantengono il loro valore per tutta l'esecuzione
- sono accessibili all'interno del blocco dove sono definite

- le variabili dichiarate statiche mantengono il valore tra due invocazioni della funzione dove sono definite
- L'inizializzazione della variabile viene fatta solamente una volta
- il programma alla destra stampa

1

2

```
int contatore(void) {
    static int c = 0; //eseguita una volta sola
    c += 1;
    return c;
}

int main (void) {
    printf("%d\n", contatore());
    printf("%d\n", contatore());
}
```

La visibilità di una variabile static è limitata al file dove è definita.

- Se il qualificatore static viene usato per una variabile globale, questa non potrà essere riferita in un altro file (tramite extern)
 - se si usa extern, verrà creata una variabile
- si può utilizzare static con una funzione per renderla visibile solamente nel file dove è definita

Vediamo il passaggio di parametri con le matrici, con alcuni esempi di codice:

- `int c[12];` definisce 12 variabili in memoria
- `c[i]` equivale a `*(c+i)`
- se passiamo `c` come parametro ad una funzione, il C lo trasforma in un puntatore ad intero
 - `int f(int a[]);` `int f(int *a);` sono equivalenti
 - Nel main si invoca con `f(c);`
- `int c[3][4];` definisce una matrice di 3 righe, ciascuna con 4 elementi
- sappiamo che `c[2]` equivale a `*(c+2*4)`; per eseguire l'operazione il C deve sapere che la matrice `c` ha 4 colonne, questa informazione deve essere mantenuta nel tipo di `c`

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

- `c[2]` equivale a `*(c+2*4)`; per eseguire l'operazione il C deve sapere che la matrice `c` ha 4 colonne
- Ricordiamo che 2 viene moltiplicato per la dimensione del tipo delle variabili che compongono l'array, in questo caso ciascuna riga punta ad un vettore di 4 elementi
- il tipo di `c` (`c[3][4]`) è `int (*p)[4]`, perché le parentesi?
- poiché `[]` ha priorità su `*`, `int *p[4]` si legge: `p` è un array di 4 elementi (ciascuno) `int *`, ovvero un array di 4 puntatori ad intero
- `int (*p)[4]` si legge: `p` è un puntatore ad un array di 4 interi

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

- `c[2][1]`: equivale a
 - `= (c[2])[1] =`
 - `= (*(c+2*sizeof(int (*) [4]))[1] =`
 - `= (*(c+2*4))[1] =` // il tipo di `c` e `c[2]` è `int (*) [4]`
 - // `*(c)` è `int[4]`
 - // se `x = *(c+2*4)` e `x` è di tipo `int[4]`
 - `= *(x+1*sizeof(int)) =`
 - `= *(x+1)`

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Cosa stampa questo codice?

```
#include <stdio.h>
int c = 2;
int main(void) {
    int a = 3;
    {
        a += 1;
        int c;
        printf("%d\n", c);
    }
}
```

Il codice stamperà "0".

In questo codice, la variabile globale `c` viene inizializzata a 2. All'interno della funzione `main`, la variabile locale `a` viene inizializzata a 3 e poi viene incrementata di 1.

All'interno del blocco delimitato da parentesi graffe, una nuova variabile locale `c` viene dichiarata e non inizializzata. Quando viene richiamata la funzione `printf`, viene stampato il valore di questa nuova variabile locale `c`, che è 0 perché non è stata inizializzata.

La variabile globale `c` non viene mai utilizzata in questo codice.

Cosa stampa questo codice?

```
#include <stdio.h>
int c = 2;
int main(void) {
    int a = 3;
    {
        a += 1;
        int c;
    }
    printf("%d,%d\n", a, c);
}
```

Il codice stamperà "4,2".

In questo codice, la variabile globale `c` viene inizializzata a 2. All'interno della funzione `main`, la variabile locale `a` viene inizializzata a 3 e poi viene incrementata di 1.

All'interno del blocco delimitato da parentesi graffe, una nuova variabile locale `c` viene dichiarata e non inizializzata. Questa variabile locale `c` viene distrutta alla fine del blocco, quindi non influisce sul valore di `c` che viene stampato dalla funzione `printf`.

La funzione `printf` stampa il valore di `a` e di `c`, ovvero 4 e 2, rispettivamente.

Definire una funzione ricorsiva che determini se esiste un percorso che permetta di attraversare un campo fiorito, dal basso verso l'alto, senza calpestare alcun fiore.

Il campo è rappresentato da una matrice, i cui valori rappresentano la presenza di un fiore (0) oppure la sua assenza (1). La posizione iniziale è fornita come parametro della funzione.

E' possibile muoversi una casella in alto oppure una casella verso destra.

```
{0,0,0,1,0},
{0,1,0,1,0},
{1,0,0,1,0},
{1,0,1,1,1},
{1,0,1,0,0}

int mossa(int dim_x, int dim_y, int campo[dim_x][dim_y], int pos_x, int pos_y) {

    if ( (pos_x<0) || (pos_y<0) || (pos_y>=dim_y) || (pos_x>=dim_x) ) {
        return 0; //mossa fuori dal percorso
    }
    if (campo[pos_x][pos_y]==0)
        return 0; // calpestato un fiore, percorso non valido
    if (pos_x==0)
        return 1; // sono arrivato nella prima riga, percorso valido
    return (
        mossa(dim_x, dim_y, campo, pos_x-1, pos_y) || //muovo di una casella // in alto
        mossa(dim_x, dim_y, campo, pos_x, pos_y+1)    // a destra
    );
}
```

Dato un array di 6 interi ed un intero `x`, stampare l'indice nell'array dell'ultima occorrenza di `x`, oppure "non trovato" se non c'è. `int x[6] = {2,6,1,5,1,5};`

```
#include <stdio.h>
```

```
int main(void) {
    int x[] = {2, 6, 1, 5, 1, 5};
    int elem = 5;
    int dim = 6;
    int index = -1;

    for (int i = dim - 1; i >= 0; i--) {
        if (x[i] == elem) {
            index = i;
            break;
        }
    }
}
```

Scritto da Gabriel

```

if (index == -1) {
    printf("non trovato\n");
} else {
    printf("L'ultima occorrenza di %d si trova all'indice %d\n", elem, index);
}

return 0;
}

```

L'esercizio su campo fiorito, sarebbe stato possibile risolverlo con una funzione ricorsiva se le mosse possibili fossero state 3: muovi di una casella a sinistra, muovi di una casella in alto, muovi di una casella a destra? Perché? {0,0,0,1,0}, {0,1,0,1,0}, {1,0,0,1,0}, {1,0,1,1,1}, {1,0,1,0,0}

- Sì, sarebbe stato possibile risolvere il problema con una funzione ricorsiva se le mosse possibili fossero state 3 (sinistra, alto e destra). La soluzione sarebbe stata simile all'attuale, ma avrebbe incluso anche la possibilità di muoversi a sinistra. Questo avrebbe reso la soluzione più completa e flessibile, poiché avrebbe permesso di considerare un maggior numero di percorsi possibili.

È possibile passare argomenti ad un programma C direttamente da linea di comando al momento dell'esecuzione (invece di leggerli da tastiera).

```
gcc -o palindroma palindroma.c
```

```
palindroma abba
```

```
stampa "la stringa abba è palindroma"
```

- i parametri sono separati da spazi: "palindromo abba 1221" indica due parametri stringa
- gli argomenti da linea di comando sono i parametri della funzione main:
int main (int argc, char *argv[])
- i nomi delle variabili sono arbitrari, ma si usa argc e argv per tradizione

```
int main (int argc, char *argv[])
```

- una volta definiti i parametri della funzione main, ovvero
int main (void) -> int main (int argc, char *argv[])
- si avrà sempre un almeno argomento passato da linea di comando

```
argc>=1
```

```
argv[0] è il nome del programma
```

- argc: numero di parametri passati dalla linea di comando (incluso il nome del programma)
- argv[i]: l'i-esimo argomento

Se volessimo passare dei numeri da linea di comando come argomenti, possiamo convertire le stringhe in numeri:

Prototipo della funzione	Descrizione della funzione
<code>double strtod(const char *nPtr, char **endPtr);</code>	Converte la stringa nPtr in un double.
<code>long strtol(const char *nPtr, char **endPtr, int base);</code>	Converte la stringa nPtr in un long.
<code>unsigned long strtoul(const char *nPtr, char **endPtr, int base);</code>	Converte la stringa nPtr in un unsigned long.

Conversione Stringa -> Double



- **double strtod(const char *nPtr, char **endPtr);**
 - nPtr è la stringa che contiene il nostro numero: i caratteri di spaziatura all'inizio della stringa vengono ignorati
 - la stringa può contenere altri caratteri dopo le cifre del numero, es. "12.4ciao", dopo l'esecuzione di strtod(), endPtr punta al primo carattere dopo il numero tradotto, nell'esempio 'c'.
 - Se nessun carattere viene tradotto endPtr punta al primo carattere di *nPtr
- ```
char *endPtr;
char *p = " 23.2abc";
printf("%f -%s-\n", strtod(p, &endPtr, endPtr); // 23.20000 -abc-
```

## Conversione Stringa -> Long



- **long strtol(const char \*nPtr, char \*\*endPtr, int base);**
  - nPtr, endPtr hanno lo stesso che hanno in strtod()
  - base è la base di decodifica del numero (binaria, decimale), solitamente base = 0
- ```
char *endPtr;
char *p = " -232abc";
printf("%f -%s-\n", strtol(p, &endPtr, 0), endPtr); // -232 -abc-
```

Altre Funzioni per Stringhe



- in <stdio.h> sono presenti anche le seguenti funzioni:
- **int sprintf(char *s, const char *format, ...);**
 - equivalente alla printf, ma invece di stampare a video, memorizza nella stringa sint
 - sprintf(char *s, const char *format, ...);**
 - può essere utilizzata per simulare l'assegnamento ad una stringa
- **int sscanf(char *s, const char *format, ...);**
 - equivalente alla scanf, ma l'input è letto dalla stringa s
 - può essere utilizzata come alternativa alle funzioni strtod(), strtol()

Scrivere un frammento di codice che stampi la lista dei parametri passati da linea di comando, escluso il nome del programma

```
int main (int argc, char *argv[]) {
    int i;
    for (i = 1; i < argc; i++) {
        printf("Parametro %d: %s\n", i, argv[i]);
    }
    return 0;
}
```

Completare la seguente funzione ricorsiva e mostrarne la correttezza
 // date n persone in una stanza che stanno salutandosi, calcolare il numero totale di strette di mano
 unsigned int handshake(unsigned int n) {
 //POST Restituisce il numero totale di strette di mano tra n persone

```

}

unsigned int handshake(unsigned int n) {
    if (n == 0 || n == 1) {
        return 0;
    } else {
        return (n - 1) + handshake(n - 1);
    }
}

```

La correttezza della funzione può essere dimostrata analizzando il problema alla base della funzione. La formula per calcolare il numero totale di strette di mano tra n persone è data da n choose 2, ovvero $n! / (2! * (n-2)!)$.

La funzione implementa questa formula utilizzando la ricorsione. Ad ogni chiamata ricorsiva, il numero di persone viene ridotto di 1 e la funzione restituisce $n-1$ strette di mano più la somma del numero di strette di mano tra le rimanenti $n-1$ persone.

La condizione di terminazione della ricorsione è quando il numero di persone è minore o uguale a 1, in questo caso la funzione restituisce 0.

Questa implementazione mostra la corretta applicazione della formula per il calcolo del numero di strette di mano e dimostra la corretta implementazione della funzione.

Vediamo qualche esempio di *divide and conquer*, dunque algoritmi che tendono a risolvere sottoproblemi sempre più piccoli e risolvibili.

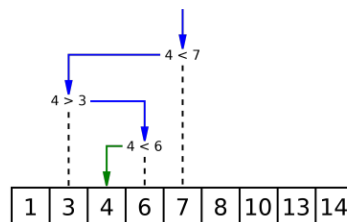
Ad esempio, la *ricerca binaria* è un algoritmo di ricerca che utilizza un approccio divide-et-impera per trovare un elemento specifico in una struttura di dati ordinata. La funzione di ricerca binaria divide continuamente il set di dati in sotto-insiemi più piccoli finché l'elemento cercato non viene identificato o il sotto-insieme è vuoto.

La ricerca binaria è importante perché è molto più efficiente rispetto alla ricerca sequenziale per grandi set di dati.

```

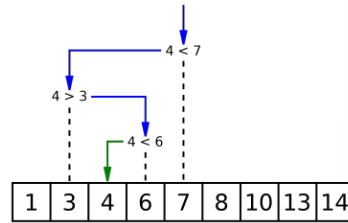
int ricerca_binaria(int *X, int dim, int elem) {
    /*
     * PRE: X ordinato in modo crescente
     * POST: restituisce 1 se elem è in X, 0
     * altrimenti
     */
    /*
     * confronta elem con l'elemento centrale
     * dell'array; se è minore elem può solo essere
     * nella metà di sinistra (X è ordinato),
     * altrimenti può solo essere a destra. Ripeti
     * l'algoritmo con la metà dell'array prescelta.
     */
}

```



```
int ricerca_binaria(int *X, int dim, int elem) {
    /* PRE: X ordinato in modo crescente
       POST: restituisce 1 se elem è in X, 0 altrimenti */
    int n = dim/2;
    if (dim<=0)
        return 0;
    if (elem==X[n])
        return 1;

    if (elem<X[n])
        return ricerca_binaria(X, n, elem);
    else
        return ricerca_binaria(X+n+1, dim-n-1, elem);
}
```



```
int ricerca_binaria(int *X, int dim, int elem) {
    /* PRE: X ordinato in modo crescente
       POST: restituisce 1 se elem è in X, 0 altrimenti */
    int n = dim/2;
    if (dim<=0) // *1
        return 0;
    if (elem==X[n])
        return 1; // *2

    if (elem<X[n])
        return ricerca_binaria(X, n, elem); // *3
    else
        return ricerca_binaria(X+n+1, dim-n-1, elem); // *4
}
```

*1 l'array vuoto non contiene elem; *2 elem trovato
 *3 ricerca_binaria()==1 => elem trovato; se ricerca_binaria()==0 elem non è in X[0],...,X[n-1], ma elem<X[n]<=X[n+1]<=...<=X[dim-1], quindi possiamo concludere che elem non sia in X e restituire 0 (*4 stesso ragionamento di *3)
 Notate che stiamo usando l'ipotesi $P(n/2) \Rightarrow P(n)$ (possiamo usare qualsiasi $n' < n$)

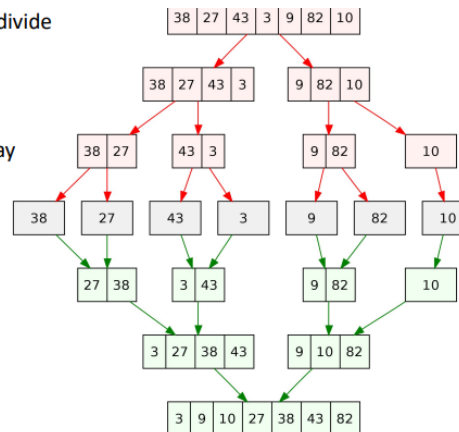
• Divide and Conquer consiste in

1. Dividere il problema in sottoproblemi più piccoli.
 2. Risolvere ricorsivamente i sottoproblemi
 3. Combinare le soluzioni dei sottoproblemi per ottenere la soluzione del problema originale
- Notare che la ricorsione è parte integrante della strategia di risoluzione del problema.

Esempio notevole: Mergesort

- Mergesort è un algoritmo che utilizza la strategia divide and conquer per ordinare un array di valori
 - Dato un array X di dimensione n,
1. richiama mergesort sulla prima metà dell'array ottenendo X₁
 2. richiama mergesort sulla seconda metà dell'array ottenendo X₂
 3. combina gli array ordinati X₁, X₂

```
MERGE-SORT(A, p, r) {
    if (p<r) {
        q = (p + r)/2;
        MERGE-SORT(A, p, q);
        MERGE-SORT(A, q + 1, r);
        combina_array(A, p, q, r);
    }
}
```



Possiamo inoltre dettagliare gli operatori di Incremento/Decremento, che Forniscono un modo compatto di esprimere espressioni di incremento: $x=x+1$.

Esempio di espressione	Spiegazione
<code>++a</code>	Incrementa a di 1, quindi usa il nuovo valore di a nell'espressione in cui si trova a.
<code>a++</code>	Usa il valore corrente di a nell'espressione in cui si trova a, quindi incrementa a di 1.
<code>--b</code>	Decrementa b di 1, quindi usa il nuovo valore di b nell'espressione in cui si trova b.
<code>b--</code>	Usa il valore corrente di b nell'espressione in cui si trova b, quindi decrementa b di 1.

```
int c = 5;
printf( "%d\n", c ); // stampa 5
printf( "%d\n", c++ ); // stampa 5 e poi postincrementa
printf( "%d\n\n", c ); // stampa 6

c = 5;
printf( "%d\n", c ); // stampa 5
printf( "%d\n", ++c ); // preincrementa e poi stampa 6
printf( "%d\n", c ); // stampa 6
```

Gli operatori di incremento e decremento

- si applicano solamente a nomi di variabili: non si può scrivere $++(x + 1)$
- hanno priorità rispetto agli altri operatori aritmetici
- permettono di scrivere codice compatto:

```
char a[10] = "hello";
int i=0;
while(i<5) {
    printf("%c ", a[i++]);
}
```

- ma, a volte, meno leggibile

```
int i=1; printf("%d\n", 3*i+++3+i); //stampa 8
```

Cenni di complessità computazionale

La complessità computazionale è la misura del tempo e delle risorse computazionali richiesti da un algoritmo per risolvere un problema in base alla dimensione dell'input. Questa disciplina studia le proprietà dei problemi algoritmici e delle soluzioni ad essi associate, classificandole in categorie come facile, difficile o impossibile da risolvere in un tempo ragionevole. La complessità computazionale è importante perché ci aiuta a comprendere i limiti delle tecnologie informatiche e a progettare sistemi efficienti per affrontare problemi complessi. Inoltre, la comprensione della complessità computazionale è fondamentale per la progettazione di algoritmi e sistemi sicuri e affidabili.

Un algoritmo è significativamente più efficiente di un altro se la sua complessità è minore.

Problema: creare una funzione che, dato un array in input, verifichi se il primo elemento di un array è uguale al secondo

- Misuriamo il tempo di esecuzione in termini del numero di istruzioni che vengono eseguite
- Il tempo di esecuzione è indipendente dalla dimensione dell'array
- Il tempo di esecuzione è costante, si scrive $O(1)$
- $O(1)$ non significa che viene eseguita esattamente una istruzione,
- $O(1)$ significa che il tempo di esecuzione è costante, ovvero che esiste una funzione costante che è un limite superiore al tempo di esecuzione
- Problema 2: creare una funzione che, dato un array in input, verifichi se il primo elemento di un array è uguale al secondo, terzo e quarto
- Tempo di esecuzione: $O(1)$

- Problema: creare una funzione che, dato un array in input e la sua dimensione, calcoli il valore massimo
- Si deve scorrere tutto l'array, confrontando ogni elemento di A con il max
- Il numero di istruzioni che vengono eseguite dipende dalla dimensione di A
- $O(1)$ non è più un limite superiore al numero di istruzioni, ma $O(n)$ lo è
- $O(n)$: esiste una costante k: $n*k$ è un limite superiore al tempo di esecuzione

```
int max_value(int A[], int n) {
    /*
     * PRE: A ha n>0 elementi
     * POST: calcola il valore massimo in A
     */
    int max = A[0];
    for(int i=1; i<n; i+=1) {
        if (A[i]>max)
            max = A[i];
    }
    return max;
}
```

- Problema: funzione che verifichi se un elemento di un array è duplicato altrove nell'array.
 - Il primo elemento deve essere confrontato con ogni altro elemento dell'array;
 - il secondo elemento deve essere confrontato con ogni altro elemento tranne il primo (esso è già stato confrontato con il primo).
 - Il terzo elemento deve essere confrontato con ogni altro elemento eccetto i primi due.
- il totale dei confronti sarà: $n-1 + n-2 + \dots + 2 + 1$
- La somma dei primi $n-1$ numeri è $n*(n-1)/2 = n^2/2 - n/2$.
- Il termine principale che influenza la complessità è quello più grande: n^2
- La complessità dell'algoritmo è perciò $O(n^2)$

Come detto, è possibile realizzare *puntatori a funzione*; il nome di un array è in realtà l'indirizzo in memoria del primo elemento dell'array, ma il nome di una funzione è in realtà l'indirizzo in memoria di partenza del codice della funzione stessa.

Possiamo assegnare ad un puntatore una funzione, passarle come parametro.

Come indichiamo un puntatore a funzione?

- `tipo_restituito (*nome_funzione)(parametri)`
- `int (*compare)(int a, int b)`

Consideriamo la funzione che ordina un array di interi selezionando ripetutamente l'elemento minore ed inserendolo ad inizio array

Alla seconda iterazione seleziona il minimo tra gli elementi dell'array che vanno dal secondo all'ultimo e lo inserisce in seconda posizione

In letteratura l'algoritmo prende il nome di Insertion Sort.

```
void insertionSort(int *X, int size) {
    int indice_min;
    for(int i=0; i<size-1; i+=1) {
        indice_min =
        trova_indice_minimo(X+i, size-i);
        scambia(X+i, &X[indice_min+i]);
    }
}
```

Se vogliamo generalizzare insertionSort in modo da poter ordinare l'array in modo crescente o decrescente, possiamo ad esempio definire la seguente funzione

passare un puntatore a funzione come parametro, assumendo che tale funzione calcoli il minimo o il massimo di un array:

```
void insertionSortGeneral(int *X,
int size, int (*seleziona_elem)(int
*A, int size));
```

```
void insertionSortGeneral(int *X, int size,
int (*seleziona_elem)(int *A, int size)) {
    int elem;
    for(int i=0; i<size-1; i+=1) {
        elem = (*seleziona_elem)(X+i, size-i);
        scambia(X+i, &X[elem+i]);
    }
}
```

Un modo compatto per creare un menu nel quale l'utente indica con un numero la funzionalità desiderata è quello di creare un array di puntatori a funzioni e poi richiamare la funzione dell'indice nell'array corrispondente alla scelta dell'utente:

```
void function1(int a);
void function2(int b);
void function3(int c);
void (*f[3])(int) = { function1, function2, function3 };
```

```
scanf("%d", &choice); (*f[choice])(choice); void function1(int a) { printf("You entered %d so function1 was
called\n", a); }
```

Scritto da Gabriel

Il calcolatore non è in grado di generare una sequenza casuale di numeri. È però in grado di generare in modo deterministico una sequenza di numeri pseudocasuale, ovvero deterministica ma che abbia le stesse proprietà di numeri casuali (es. distribuzione uniforme)

Le funzioni per generare numeri casuali sono definite in <stdlib.h>, quali:

- rand() restituisce un intero x . $0 \leq x \leq \text{RAND_MAX}$ (generalmente almeno 2^8)
- la sequenza di numeri casuali dipende dal valore passato alla funzione srand()
- unsigned int seed; srand(seed);
- se utilizziamo lo stesso valore di seed, otterremo la stessa sequenza di valori casuali

Simulare il tiro di un dado 100 volte e stampare le frequenze dei valori ottenuti in C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define NUM_ROLLS 100
#define NUM_SIDES 6

int main(void) {
    int i, result, frequency[NUM_SIDES + 1] = {0};

    srand(time(NULL));

    for (i = 0; i < NUM_ROLLS; i++) {
        result = rand() % NUM_SIDES + 1;
        ++frequency[result];
    }

    printf("Side\tFrequency\n");
    for (i = 1; i <= NUM_SIDES; i++) {
        printf("%d\t%d\n", i, frequency[i]);
    }

    return 0;
}
```

Il codice utilizza la funzione rand per generare un numero casuale che rappresenta il risultato del lancio di un dado. La funzione srand viene utilizzata per inizializzare il generatore di numeri casuali con un valore univoco basato sul tempo corrente. La variabile frequency viene utilizzata per tenere traccia delle frequenze dei risultati dei lanci. Alla fine, il codice stampa i risultati sotto forma di una tabella.

Struct ed union, enum, file in C, operazioni sui file/file binari, allocazione della memoria

Le strutture (o “aggregati”) in C sono collezioni di variabili collegate sotto un unico nome

- Possono contenere variabili di tipi differenti (contrariamente agli array che contengono solo elementi dello stesso tipo)
- Le strutture sono comunemente usate per definire record di dati da memorizzare in file
- I puntatori e le strutture facilitano la creazione di strutture dati complesse (come ad es. liste, code, pile, alberi...)

Esse sono tipi di dati derivati, costituite usando elementi (membri) di altri tipi, normalmente definite con la parola chiave *struct*.

■ Esempio:

```
struct studente {  
    char cognome[20];  
    char nome[20];  
    unsigned int matricola;  
    unsigned int annoNascita;  
    char gender;  
};
```

Le variabili dichiarate
entro le parentesi sono i
membri della struttura

Il tipo dei membri di una struct può essere qualsiasi:

- Scalare (int, float, char, ...)
- Aggregato (vettori, stringhe, altre struct, union, ...)

■ Esempio:

```
struct punto {  
    int x;  
    int y;  
};  
struct rettangolo {  
    struct punto altoSx;  
    struct punto bassoDx;  
};
```

Le definizioni di strutture non riservano alcuno spazio di memoria ma creano un nuovo tipo di dati.

Le variabili di tipo struttura si dichiarano come tutte le altre variabili di tipo “standard” (i.e. int, float, char, etc.)

```
struct studente stud, inf21_22[180], *studPtr;
```

←
Esempio n. 1

```
struct studente {  
    char cognome[20];  
    char nome[20];  
    unsigned int matricola;  
    unsigned int annoNascita;  
    char gender;  
} stud, inf21_22[180], *studPtr;
```

← Esempio n. 2

*Dichiarazione accorpata
con definizione di struttura*

Le strutture possono essere inizializzate usando liste di inizializzatori (espressioni costanti), così come gli array.

■ Ad esempio:

```
struct studente stud = {"Turing", "Alan", 1001110};
```

- Cosa succede?
 - Viene creata la variabile `stud` di tipo `struct studente`
 - Si inizializzano i membri `cognome` a "Turing", `nome` ad "Alan" e `matricola` a "1001110"
- Se nella lista vi sono *meno* inizializzatori dei membri della struttura, i restanti sono inizializzati a 0 (o NULL)

Si usano due operatori per accedere ai singoli membri:

- L'operatore di membro di struttura: `.`
- L'operatore di puntatore a struttura: `->`

■ Vediamo due esempi:

```
printf("%s", stud.cognome); //stampa Turing
```

(Supponendo che `studPtr` sia stato inizializzato con l'indirizzo di `stud`)

```
printf("%s", studPtr->cognome); //stampa Turing
```

- Nota: l'espressione `studPtr->cognome` è del tutto equivalente a `(*studPtr).cognome`
 - L'operatore `.` ha infatti precedenza rispetto all'operatore `*`
 - Questo significa che `*studPtr.cognome` equivarrebbe a `*(studPtr.cognome)`, ossia la variabile puntata dal membro `cognome` che dovrebbe essere quindi un puntatore

Una variabile `struct` può essere assegnata ad un'altra (dello stesso tipo) usando l'operatore `=`

- L'assegnazione avviene mediante copia del contenuto (i.e. non del puntatore)
- Questo vale anche per le stringhe (se uno dei membri della struttura è una stringa, viene copiata l'intera stringa e non il suo puntatore), per gli array, etc.

È buona regola combinare la definizione di strutture con `typedef` per maggiore leggibilità.

■ Ad esempio:

```
struct studente {
    char cognome[20];
    char nome[20];
    unsigned int matricola;
    unsigned int annoNascita;
    char gender;
};
typedef struct studente Studente;
Studente stud;
```

In sostanza `typedef` viene usato per assegnare nomi simbolici (alias) a tipi di dato.

Solitamente struct e typedef vengono usati assieme, ma un esempio classico di utilizzo di typedef è anche il seguente:

```
typedef unsigned char Byte;
```

- Vi è una certa somiglianza tra typedef e #define (entrambi permettono di definire alias / nomi simbolici), tuttavia:
 - o typedef è limitato alla definizione di alias per tipi
 - o #define è “più generale” (ad es. possiamo definire alias per valori) ed è processato dal pre-processore, mentre le istruzioni typedef sono processate dal compilatore

I membri di una struttura sono allocati contigualmente in memoria, seguendo lo stesso ordine di definizione

- Tra un membro ed il successivo possono esserci spazi intermedi di “allineamento” della memoria
 - o Tali spazi non sono indirizzabili e hanno contenuto indefinito
 - o La presenza di spazi intermedi dipende dal microprocessore (molti richiedono che le variabili siano allocate a indirizzi multipli di un certo valore, ad esempio 4 byte, 8 byte, ...)

In tipi possono occupare un numero diverso di byte a seconda dell’architettura / compilatore.

Un operatore molto utile (ed usato) in C è *sizeof*.

Si tratta di un operatore unario che restituisce il numero di bytes necessario ad allocare un tipo o una variabile.

```
sizeof(int); // 4
```

```
char nome[20];  
sizeof(nome); // 20
```

Essendo a tutti gli effetti nuovi tipi, possiamo usare le strutture come parametri di funzione, perciò il passaggio delle variabili avviene per valore (a meno che non si effettui il passaggio di un puntatore). Una funzione può anche restituire una variabile di tipo struct.

Nota: se i parametri / valore restituito sono struct, può esserci un overhead significativo a run time (può essere molto pesante).

Perché chiamante e funzione chiamata “riconoscano” la stessa struct, la definizione deve essere unica.

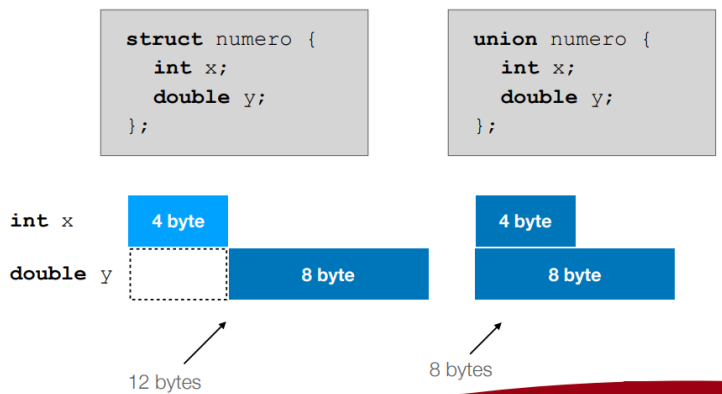
- Nel caso (frequente) di progetti con file multipli, la definizione della struttura deve essere ripetuta in tutti i file
 - o Tipicamente questa viene inserita in un file di header
 - o Più precisamente le definizioni della struttura devono essere tra loro compatibili
 - o Ciò significa che devono avere stesso numero di membri, con lo stesso nome, nello stesso ordine e con tipi tra loro compatibili

Come le strutture, un’unione è un tipo di dati derivato.

- Una unione viene definita con la parola chiave: *union*
- Contrariamente a quanto avviene nelle strutture, tutti i membri di una unione condividono la stessa memoria
- I membri di una unione possono essere di qualunque tipo, ed il numero di byte utilizzati per memorizzare una unione è pari a quello dell’elemento più grande

Strutture e unioni sono quindi molto simili tra loro, ma le unioni hanno lo scopo di evitare l’eventuale spreco di memoria causato da membri non utilizzati.

■ Vediamo un esempio:



Il C consente di specificare il numero di bit con cui memorizzare un membro `int` o `unsigned int`.

- Questo costrutto viene chiamato campo di bit
- Può essere definito sia per strutture che per unioni
- L'obiettivo dei campi di bit è quello di ottimizzare / minimizzare il numero di bit necessari per rappresentare il dato

I campi di bit si specificano all'interno della definizione della struttura col simbolo `:` seguito da `<num_bit>`

■ Vediamo un esempio:

```

struct cartaBit {
    unsigned int valore: 4;
    unsigned int seme: 2;
    unsigned int colore: 1;
};

```

- La costante intera `<num_bit>` rappresenta la larghezza del campo (ed il numero di bit si basa sull'intervallo di valori voluto)
- Avremo quindi un numero tra 0 e il numero max di bit necessari per memorizzare un `int` (ad es. 32, ma può variare a seconda dell'architettura specifica)

I campi di bit sono accorpati in modo da costituire gruppi di byte chiamati *storage unit*:

- Ciascuna storage unit è pari alla dimensione di un `int`
- L'ordine di memorizzazione dei campi nelle storage unit dipende dal particolare compilatore / implementazione
- In generale non è quindi possibile determinare con esattezza la posizione di un campo di bit: pertanto non è possibile definire un puntatore diretto a campo di bit
- Si possono definire campi anonimi (senza assegnare un nome al campo) che possono essere utilizzati per "riempire" i bit di una storage unit e forzare la memorizzazione del membro seguente all'inizio della successiva storage unit

■ Vediamo un esempio con campo di bit anonimo:

```

struct cartaBit {
    unsigned int valore: 4;
    unsigned int seme: 2;
    unsigned int : 26;
    unsigned int colore: 1;
};

```

■ D: quanti byte occupa una variabile `cartaBit`? R: **8**

■ Nota: si può forzare l'allineamento al prossimo membro così:

```

unsigned int : 0;

```

Finora abbiamo visto come accedere a file di testo in modalità sequenziale. L'accesso ai dati però è inefficiente, e in molti casi vorremmo trattare dati grezzi e non solo testo

- I file di testo sono formati da righe/stringhe di caratteri (ad es. con codifica ASCII) terminate dal simbolo '\n'
- Un'alternativa è quindi quella dei file binari: un file binario è, in pratica, una generica sequenza di byte "grezzi"

Nei file binari si dovrebbe parlare più propriamente di byte, anche se spesso viene usato il termine carattere. Generalmente i byte vengono raggruppati in blocchi di byte chiamati record, in cui ognuno viene letto/scritto in un'unica operazione. Quando i record hanno la stessa dimensione, l'accesso al record n-esimo è molto veloce perché (se il record occupa L byte) la sua posizione sarà $n * L$; in questo caso si parla di file ad accesso diretto (o casuale).

I file binari si aprono con `fopen()` ma indicando i modi binari (simili ai file di testo ma con lettera 'b')

- "rb", "wb", "ab", "r+b" o "rb+", "w+b" o "wb+", "a+b" o "ab+"
 - La differenza tra "r+b" e "w+b" è solo nell'apertura (come nel caso dei file di testo)

Anche i file binari si chiudono con `fclose()`.

Parliamo ora delle enumerazioni, che sono un insieme di costanti intere rappresentate da identificatori.

Vengono definite con la parola chiave `enum` e si definiscono in modo simile a strutture/unioni.

Fondamentalmente, avere una `enum` equivale a usare delle `#define` multiple.

■ Vediamo un esempio:

```
enum giorni{Lun, Mar, Mer, Gio, Ven, Sab, Dom};
```

■ Il tipo enumerativo può poi essere utilizzato per dichiarare nuove variabili di questo tipo

```
enum giorni gs;
```

```
enum giorni{Lun, Mar, Mer, Gio, Ven, Sab, Dom};
```

≈

```
#define Lun 0
#define Mar 1
#define Mer 2
#define Gio 3
#define Ven 4
#define Sab 5
#define Dom 6
```

In merito ai valori:

- Se non assegnati esplicitamente, i valori corrispondenti alle costanti elencate nella enumerazione partono da 0
- In generale una costante non assegnata in posizione i+1 assume il valore della costante i-esima + 1

```
enum giorni{Lun, Mar, Mer, Gio, Ven, Sab, Dom};
           0   1   2   3   4   5   6
```

■ Vediamo una definizione "alternativa":

```
enum giorni{Lun=1, Mar=2, Mer=3, Gio=4, Ven=5,
Sab=6, Dom=0};
```

Ora parliamo dei file, visti in C come uno stream (flusso) sequenziale di byte:

- I file non terminano con un carattere "speciale" ma alcune funzioni restituiscono il valore EOF (end-of-file)
- Un file può essere "paragonato" ad una lunga stringa; il file viene prima aperto, associato ad uno stream, e infine chiuso

Tre stream standard vengono aperti automaticamente quando inizia l'esecuzione di un programma:

- Lo standard input (che riceve input dalla tastiera)
- Lo standard output (che stampa output a schermo)
- Lo standard error (che stampa messaggi di errore a schermo)

Gli stream forniscono un "canale di comunicazione" tra programmi e file:

- ad esempio, lo stream standard input permette ad un programma di leggere dati dalla tastiera
- lo stream standard output permette ad un programma di stampare dati sullo schermo

Possiamo quindi dire che il sistema di I/O del C crea un livello di astrazione fra programma e dispositivo

- Lo stream è questa astrazione e il dispositivo su cui avvengono le operazioni è chiamato file

Per accedere ad un dato specifico occorre posizionarsi al byte esatto in cui inizia il dato stesso.

- File ad accesso sequenziale
 - o Hanno blocchi di informazione (record) di lunghezza variabile
 - o Per accedere al record n, occorre leggere tutti i precedenti n-1
- File ad accesso diretto (o casuale)
 - o Hanno record di lunghezza fissa
 - o La lunghezza L del record viene fissata dal programmatore
 - o Per accedere al record n, il byte d'inizio corrisponderà a $n * L$
- File di testo
 - o Sono sequenze di caratteri organizzate in righe
 - o Ogni riga è terminata da un carattere speciale di ritorno a capo (in C le funzioni I/O per stringhe e caratteri usano '\n')
- File binari
 - o Sono sequenze "grezze" di byte

L'apertura di un file restituisce un puntatore a struttura FILE (definita in stdio.h), in cui vengono salvate le informazioni e caratteristiche.

- Standard input, output ed error, sono manipolati usando stdin, stdout e stderr

La libreria standard fornisce varie funzioni di lettura e scrittura su file:

- fgetc e fputc (simili a getchar e putchar)
- fgets e fputs (per leggere/scrivere una riga da file)
- fscanf e fprintf (analoghi a scanf e printf)
- fread e fwrite

Ogni accesso a un file avviene attraverso una variabile di tipo FILE che si riferisce a tale struttura.

- Concretamente occorre quindi dichiarare un puntatore a FILE (ad es. FILE *fp;)
- La struttura FILE contiene varie informazioni "di sistema"; tra queste è molto importante il file position pointer che memorizza il punto (offset) del prossimo byte da leggere o da scrivere

Una volta dichiarata una variabile puntatore a FILE occorre creare una "linea di comunicazione" col file

- La funzione fopen permette di aprire il file associandolo ad un puntatore a struttura FILE
- La funzione fopen ha due argomenti:
 - o una stringa contenente il nome (indirizzo) del file
 - o una stringa che indica la modalità di apertura del file stesso
- Terminata l'elaborazione il file verrà chiuso con fclose

■ Vediamo un esempio:

```
FILE *fp;  
fp = fopen("studenti.txt", "r"); //aperto in lettura
```

- Se il file non può essere aperto il puntatore sarà uguale a NULL
- Nella pratica è quindi buona regola verificare sempre che l'apertura del file sia andata a termine nel modo corretto

```
FILE *fp;
fp = fopen("studenti.txt", "r"); //aperto in lettura
if (fp==NULL) {
    fprintf(stderr, "Il file non può essere aperto\n");
    return EXIT_FAILURE;
} else {
    ...
    fclose(fp);
}
```

■ Ci sono diverse modalità di apertura del file:

r	Apri un file esistente per la lettura.
w	Crea un file per la scrittura. Se il file esiste già, <i>elimina</i> i contenuti correnti.
a	Apri o crea un file per scrivere alla fine del file – cioè, le operazioni di scrittura aggiungono dati al file.
r+	Apri un file esistente per l'aggiornamento (lettura e scrittura).
w+	Crea un file per l'aggiornamento. Se il file esiste già, <i>elimina</i> i contenuti correnti.
a+	Append: apri o crea un file per l'aggiornamento; tutta la scrittura è effettuata alla fine del file – cioè, le operazioni di scrittura aggiungono dati al file.
rb	Apri un file esistente per la lettura in forma binaria.
wb	Crea un file per la scrittura in forma binaria. Se il file esiste già, <i>elimina</i> i contenuti correnti.
ab	Append: apri o crea un file per la scrittura alla fine del file in forma binaria.
rb+	Apri un file esistente per l'aggiornamento (lettura e scrittura) in forma binaria.
wb+	Crea un file per l'aggiornamento in forma binaria. Se il file esiste già, <i>elimina</i> i contenuti correnti.
ab+	Append: apri o crea un file per l'aggiornamento in forma binaria; la scrittura è effettuata alla fine del file.

Nota: l'apertura di un file in modalità aggiornamento permette sia la lettura che scrittura dello stesso.

Ci sono diverse modalità in aggiornamento:

- “r+”: lettura/scrittura; il file deve esistere
 - “w+”: lettura/scrittura; crea il file vuoto
 - “a+”: lettura/scrittura al fondo; se il file non esiste lo crea, e aggiunge alla fine se invece già esiste
- La differenza tra “r+” e “w+” è solo nell'apertura.

Ogni accesso (lettura o scrittura) avviene in modo sequenziale a partire dalla posizione di offset corrente. Nei file di testo le funzioni di I/O sono analoghe a quelle per le stringhe (salvo indicazione del puntatore a FILE):

```
fscanf(fp, <stringa_formato>, <espressioni>);
fprintf(fp, <stringa_formato>, <espressioni>);
char c = fgetc(fp);
fputc(<carattere>, fp);
char* s[20]; fgets(s, 20, fp);
fputs(s, fp);
```

Le operazioni su file (cioè su disco) sono lente. I dati diretti a file di output sono perciò accumulati temporaneamente in memoria (in uno spazio di buffer).

- Quando il buffer è pieno, o si esegue una fclose, il buffer viene svuotato ed i dati sono inviati tutti assieme (flush)

Scritto da Gabriel

- Possiamo anche forzare una operazione di flush del buffer associato al file puntato da `fp` con:
`fflush(fp);`

Listiamo altre funzioni su file:

`remove(<filename>);` cancella il file indicato dalla stringa *filename* e restituisce 0 se ha esito positivo

`rename(<file1>,<file2>);` rinomina *file1* in *file2* e restituisce 0 se ha esito positivo

`freopen(<file>,<modo>,fp);` associa uno stream pre-esistente ad un altro file

- Il file con nome `<file>` viene associato al puntatore `fp`; se già aperto lo stream pre-esistente di `fp` viene chiuso
- La stringa `<modo>` indica la modalità di apertura (funziona in modo analogo a `fopen`)
- Se ha successo restituisce un puntatore a `FILE`, altrimenti `NULL`

`rewind(fp);` riporta l'offset del file `fp` all'inizio in modo che la prossima operazione di I/O inizi da 0

`fseek(fp,<offset>,<posizione>);` sposta l'indicatore di posizione del file puntato da `fp` a `<posizione>` e si sposta di `n bytes = <offset>`

- `int <offset>`: numero di bytes da posizione
- `int <posizione>`: può assumere tre valori indicati dalle costanti `SEEK_END`, `SEEK_SET`, `SEEK_CUR`
- Restituisce un intero: 0 se ha successo, un numero diverso da 0 altrimenti

■ `rewind()` vs `fseek()`:

- La funzione `rewind()` può essere sostituita da `fseek()` (che è più generale); questa istruzione equivale a `rewind()`:
`fseek(fp, 0L, SEEK_SET);`
- `ftell(fp);` restituisce il valore attuale (come `long int`) dell'indicatore di posizione del file puntato da `fp`

Finora abbiamo visto come accedere a file di testo in modalità sequenziale. L'accesso ai dati però è inefficiente, e in molti casi vorremmo trattare dati grezzi e non solo testo

- I file di testo sono formati da righe/stringhe di caratteri (ad es. con codifica ASCII) terminate dal simbolo `'\n'`
- Un'alternativa è quindi quella dei file binari: un file binario è, in pratica, una generica sequenza di byte "grezzi"

Nei file binari si dovrebbe parlare più propriamente di byte, anche se spesso viene usato il termine carattere

- Generalmente i byte vengono raggruppati in blocchi di byte chiamati record
- Ogni record viene letto/scritto in un'unica operazione

Quando i record hanno la stessa dimensione, l'accesso al record *n*-esimo è molto veloce perché (se il record occupa *L* byte) la sua posizione sarà *n*L*

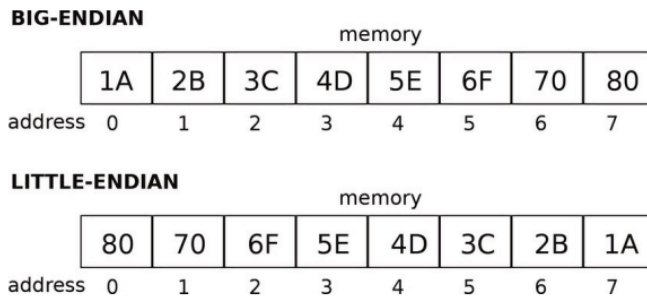
- In questo caso si parla di file ad accesso diretto (o casuale)

Similmente ai file di testo, anche i file binari si aprono con `fopen()` e chiudono con `fclose()`, solo che abbiamo le stesse direttive ma con la lettera "b" (binary).

- "rb", "wb", "ab", "r+b" o "rb+", "w+b" o "wb+", "a+b" o "ab+"
 - La differenza tra "r+b" e "w+b" è solo nell'apertura (come nel caso dei file di testo)

I file binari possono avere problemi di portabilità tra architetture (causati da diverse strategie di memorizzazione).

Ad es. codifica *big-endian* vs *little-endian*:



Per le operazioni di lettura, dettagliamo funzione ed esempi.

La lettura di blocchi di byte si ottiene con la funzione `fread(p, <dim_record>, <num_record>, fp)` ;

- Legge dal file `fp` un numero di record pari a `<num_record>` ciascuno di `<dim_record>` byte e li colloca nella struttura puntata da `p`
- Restituisce `size_t` che contiene il numero di record letti
- Fa avanzare il *file position pointer* del numero di byte letti

■ Vediamo un primo esempio:

```
struct studente stud;
fp = fopen("studenti.dat", "rb");
fread(&stud, sizeof(struct studente), 1, fp);
```

■ Vediamo un altro esempio:

```
struct studente inf21_22[180];
fp = fopen("studenti.dat", "rb");
fread(inf21_22, sizeof(struct studente), 180, fp);
```

Dettagliamo ora, con la stessa struttura, le operazioni di scrittura.

La scrittura di blocchi di byte si ottiene con la funzione `fwrite(p, <dim_record>, <num_record>, fp)` ;

- Scrive dal file `fp` un numero di record pari a `<num_record>` ciascuno di `<dim_record>` byte, prelevandoli dalla struttura (o array) puntata da `p`
- Restituisce `size_t` che contiene il numero di record scritti; può essere un numero inferiore a `<num_record>` in caso di errore in scrittura
- Fa avanzare il *file position pointer* del numero di byte scritti

■ Vediamo un esempio:

```
struct studente stud;
...
fwrite(&stud, sizeof(struct studente), 1, fp);
```

■ E se volessi scrivere su file due volte i dati dello stesso `stud`?

```
struct studente stud;
...
fwrite(&stud, sizeof(struct studente), 1, fp);
fwrite(&stud, sizeof(struct studente), 1, fp);
```

■ Domanda: potrei scrivere la stessa istruzione come segue?

```
...
fwrite(&stud, sizeof(struct studente), 2, fp);
```

Risposta: **No!!!**

Ricapitolando quanto detto:

- Le funzioni `fread` e `fwrite` leggono e scrivono un numero di byte pari a *
- Le funzioni ricevono un semplice puntatore al blocco e non hanno alcuna informazione sul contenuto del blocco stesso
- Solo quando il blocco viene letto da `fread` e viene fatto un cast (anche implicito) assegnandolo ad una variabile, esso assume un significato specifico

Vediamo un esempio:

```
struct persona {
    char nominativo[30];
    unsigned int annoNascita;
} iscritto;
fread(&iscritto, sizeof(iscritto), 1, fp);
```

- Solo quando il blocco viene letto e collocato in `iscritto` che il suo contenuto è identificabile come variabile `nominativo` di tipo puntatore a `char` (stringa di 30 caratteri) e come variabile `annoNascita` di tipo `unsigned int`

Parliamo invece del posizionamento nei file binari.

Come per i file di testo, le funzioni `fread` e `fwrite` posizionano il file position pointer (offset) all'inizio del record successivo

Il posizionamento generico si ottiene con la funzione `fseek(fp, <offset>, <origine>);`

- `offset` (di tipo `long`): indica il numero di caratteri/blocchi in cui deve essere spostato il file position pointer rispetto ad `<origine>` (nota: può essere un valore negativo)
- `origine`: indica la posizione da cui calcolare l'offset:
 - `SEEK_SET`: da inizio file
 - `SEEK_CURR`: dalla posizione corrente
 - `SEEK_END`: dalla fine del file (offset negativo)

Ad esempio:

```
fseek(fp, 10, SEEK_SET);
```

- Sposta il *file position pointer* al 11° blocco (il 1° ha posizione 0)

Per conoscere la posizione del *file position pointer* si può usare la funzione `ftell`:

```
pos = ftell(fp);
```

- `pos` è una variabile di tipo `long` (in caso di errore la funzione `ftell` restituisce -1L)

In alternativa si possono usare `fgetpos` e `fsetpos`

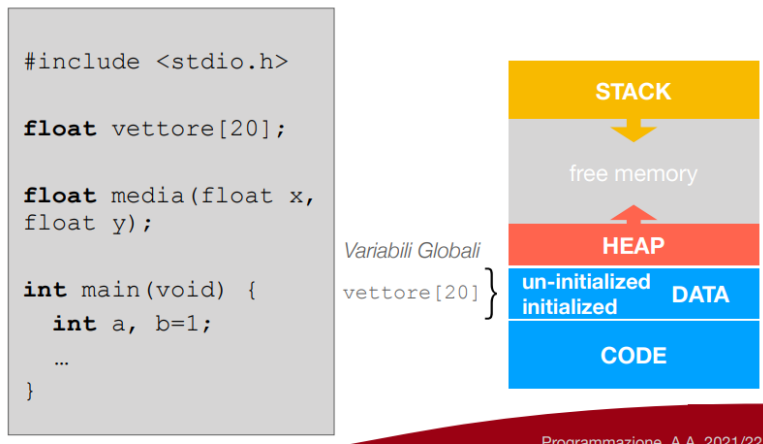
Come detto più volte, le operazioni di I/O su file sono estremamente lente. I file infatti sono un classico esempio di memoria di storage (archivio) e non di lavoro.

- Conviene quindi limitare il più possibile l'accesso a file effettuando le operazioni di I/O strettamente necessarie
- Solitamente si caricano i dati in memoria (in opportune strutture dati) e si aggiorna il file solo alla fine dell'elaborazione dei dati

Il termine allocazione della memoria viene utilizzato per indicare l'assegnazione di un blocco di memoria RAM. La memoria RAM può essere allocata:

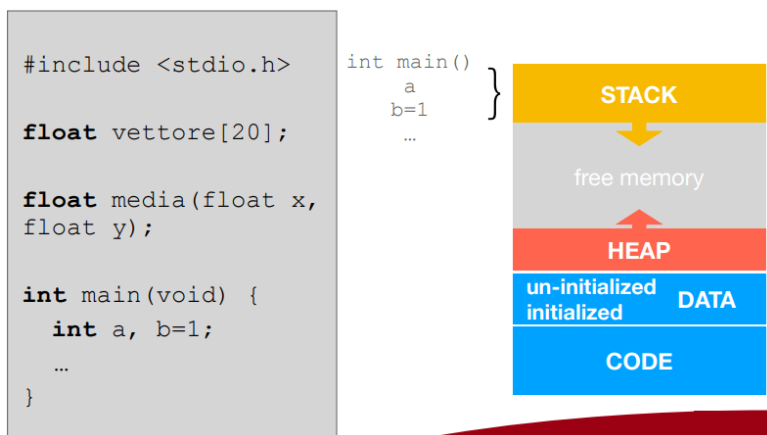
- Staticamente
 - o Si ha principalmente per le variabili di tipo globale (e per le variabili definite static)
 - o Il blocco di memoria viene definito al momento della compilazione del codice
 - o La memoria (nel segmento DATA) viene allocata prima dell'avvio del programma
 - o Nota bene: a run-time la dimensione del blocco non è più modificabile ed il blocco di memoria non viene mai rilasciato

■ Vediamo un esempio:



- Automaticamente
 - o Si ha quando in una funzione (anche il main) viene definita una variabile locale o un parametro
 - o Anche in questo caso il blocco di memoria viene definito al momento della compilazione del codice
 - o La memoria viene allocata nello STACK a run-time
 - o Nota bene: a run-time la dimensione del blocco non è più modificabile ed il blocco viene rilasciato automaticamente al termine dell'esecuzione della funzione

■ Vediamo un esempio:



L'allocazione automatica della memoria può avvenire con una struttura apposita definita come *stack*, oppure pila, struttura dati LIFO (Last In First Out). Esso è costituito da blocchi contigui di memoria allocati in modo lineare a partire da indirizzi di memoria alti. Ha una dimensione pre-fissata e (relativamente) limitata che può causare errori di stack overflow.

- Dinamicamente
 - o L'allocazione dinamica della memoria è sotto il controllo del programmatore ed avviene mediante funzioni apposite
 - o Il blocco di memoria in questo caso viene sia definito che allocato a run-time; l'allocazione di memoria è nello HEAP
 - o La dimensione del blocco è in buona sostanza un argomento della funzione di allocazione
 - o Attenzione: a run-time la dimensione è modificabile e il blocco è rilasciato solo esplicitamente mediante la chiamata di una funzione apposita

Riguardo all'allocazione dinamica, il nucleo centrale del sistema di allocazione dinamica della memoria in C è costituito dalle funzioni:

- malloc(): alloca un blocco di memoria libera

■ Il prototipo della funzione `malloc()` è il seguente:

- `void *malloc(size_t <num_byte>)`
- La funzione alloca un blocco di byte (non inizializzato) formato da `<num_byte>` byte e restituisce un puntatore void al primo byte della memoria allocata
- Se la memoria nello HEAP non è sufficiente restituisce NULL
- Vediamo un esempio:

```
int *p;
p = (int*)malloc(sizeof(int));
```

- Il compilatore considererà i `sizeof(int)` byte puntati da `p` come un intero
- Nota: il compilatore non sa l'estensione della variabile `p`

■ Vediamo un secondo esempio:

```
int *p;
p = (int*)malloc(sizeof(int)*20);
```

- Concettualmente abbiamo appena dichiarato un vettore di 20 interi, in modo analogo alla dichiarazione:

```
int p[20];
```

- Nel primo dei due casi però, il compilatore non ha modo di sapere la reale estensione dell'array `p`; la sua corretta gestione è quindi demandata al solo programmatore

■ In generale il cast esplicito da `void*` non è necessario

- L'assegnazione di un puntatore void ad un puntatore non-void è lecita e l'assegnazione effettua un cast implicito
- Sarebbe quindi ugualmente corretto:

```
int *p;
p = malloc(sizeof(int));
```

- Tuttavia, rendere il cast esplicito rende più leggibile il codice

■ Nota bene: non si può usare l'operatore `sizeof` per una variabile (blocco di memoria) allocato dinamicamente

- `free()`: libera la memoria precedentemente allocata
- Tali funzioni sono definite nella libreria `<stdlib.h>`

L'allocazione dinamica è fondamentale nella pratica (spesso le necessità di memoria non sono note a priori)

- Questo vale innanzitutto per le variabili / strutture dati "semplici"
- Ma è sostanzialmente inevitabile per le strutture dati più avanzate (liste, alberi, grafi, ...)

Oltre a `malloc()`, ci sono altre funzioni di allocazione dinamica della memoria:

- `calloc`: ha prototipo `void *calloc(size_t, size_t)` ed alloca una quantità di memoria pari a `num*size` (ossia tali da contenere un array di elementi, ciascuno di bytes)
- `realloc`: ha prototipo `void *realloc(void*, size_t)` e ridimensiona a byte il blocco di memoria puntato da `ptr` (creato con una `malloc/calloc`), preservandone il contenuto

Liste collegate con puntatori (linked lists), alberi, alberi binari di ricerca

Una lista è una successione finita di elementi di un tipo.

L'informazione codificata dalla lista riguarda:

- La successione di elementi (valori)
- La relazione di ordine tra gli elementi stessi

La lista è qualificata non solo dai valori che rappresenta ma anche dalle operazioni che ci si eseguono:

- Inserimento e cancellazione (in testa, coda, intermedia)
- Visita / ricerca
- Inizializzazione

Possiamo ottenere diverse tipologie di lista definendo diverse modalità di inserimento/cancellazione elementi:

- pila (stack): è una struttura di tipo LIFO, i.e. "last in first out"
- coda (queue): è una struttura di tipo FIFO, i.e. "first in first out"

Una lista può essere rappresentata:

- In forma sequenziale: gli elementi sono rappresentati in un array e il loro ordine è codificato in modo implicito dalla posizione
- In forma collegata: in questo caso la relazione è resa esplicita e ogni elemento è associato all'informazione che identifica il successore (questa può essere un indice o un puntatore)

Una lista collegata con puntatori è una successione di elementi (nodi) connessi da puntatori.

Le liste collegate sono strutture dinamiche e la loro dimensione può aumentare/diminuire a run time.

Gli array invece hanno dimensione pre-fissata e possono riempirsi.

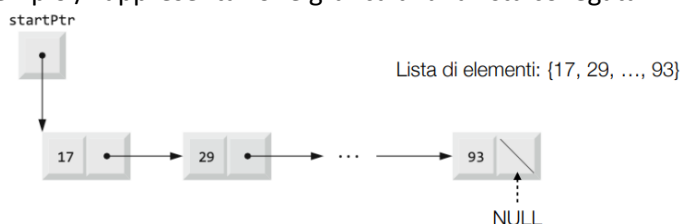
- Gli elementi di un array sono memorizzati in modo contiguo;
 - o questo permette l'accesso immediato ad un elemento
 - o ma le inserzioni o cancellazioni di un elemento sono lente
- Gli elementi di una lista collegata, seppur "logicamente" in sequenza, occuperanno indirizzi di memoria non contigui

Le liste collegate non offrono un accesso immediato ai loro elementi, ma inserimento/cancellazione di elementi possono seguire diverse strategie.

- L'elemento atomico di una lista è il *nodo* che può essere definito da una struttura "autoreferenziale":

```
struct nodoLista {
    int dato;
    struct nodoLista *nextPtr;
};
```

Esempio / rappresentazione grafica di una lista collegata:



```
struct nodoLista {
    int dato;
    struct nodoLista *nextPtr;
};
```

L'utilizzo di liste collegate richiede la definizione di alcune funzioni di base:

- inizializzazione
- visita (solitamente stamperà i valori dei nodi)
- ricerca di un elemento
- inserimento
- cancellazione

Vediamo un insieme di funzioni:

```
#include <stdio.h>
#include <stdlib.h>

struct nodoLista {
    int valore;
    struct nodoLista *nextPtr;
};

typedef struct nodoLista Lista;

void init(Lista **ptrPtr);
int isempty(Lista *ptr);
void print_list(Lista *ptr);
void print_list_rec(Lista *ptr);
void pre_insert(Lista **ptrPtr, int val);
void suf_insert(Lista **ptrPtr, int val);
void suf_insert_rec(Lista **ptrPtr, int val);
void ord_insert(Lista **ptrPtr, int val);

int main(void) {

    //Lista *pila = NULL;
    Lista *pila;
    init(&pila);
    printf("Lista vuota? %d\n", isempty(pila));

    suf_insert_rec(&pila, 17);
    printf("Lista vuota? %d\n", isempty(pila));
    suf_insert_rec(&pila, 29);
    suf_insert_rec(&pila, 5);
    //suf_insert(&pila, 32);
    //suf_insert(&pila, 93);
    print_list(pila);
    print_list_rec(pila);

    return 0;
}

void init(Lista **ptrPtr) {
    *ptrPtr = NULL;
}

// restituisce 1 se la lista è vuota, 0 altrimenti
```

Scritto da Gabriel

```
int isempty(Lista *ptr) {
    return ptr == NULL;
}

void print_list(Lista *ptr) {
    while(ptr != NULL) {
        printf("%d ", ptr->valore);
        ptr = ptr->nextPtr;
    }
    printf("\n");
}

void print_list_rec(Lista *ptr) {
    if(ptr != NULL) {
        printf("%d ", ptr->valore);
        //ptr = ptr->nextPtr;
        print_list_rec(ptr->nextPtr);
    }
    printf("\n");
}

// inserimento in testa
void pre_insert(Lista **ptrPtr, int val) {
    Lista *tmpPtr;

    tmpPtr = *ptrPtr;
    *ptrPtr = malloc(sizeof(Lista));
    (*ptrPtr)->valore = val;
    (*ptrPtr)->nextPtr = tmpPtr;
}

// inserimento in coda
void suf_insert(Lista **ptrPtr, int val) {
    while(*ptrPtr != NULL) {
        ptrPtr = &((*ptrPtr)->nextPtr);
    }
    pre_insert(ptrPtr, val);
}

// inserimento in coda con ricorsione
void suf_insert_rec(Lista **ptrPtr, int val) {
    if(*ptrPtr != NULL) {
        suf_insert_rec(&((*ptrPtr)->nextPtr), val);
    }
    else {
        pre_insert(ptrPtr, val);
    }
}

// inserimento in coda
void ord_insert(Lista **ptrPtr, int val) {
    while(*ptrPtr != NULL && (*ptrPtr)->valore < val) {
        ptrPtr = &((*ptrPtr)->nextPtr);
    }
}
```

```

}
pre_insert(ptrPtr, val);
}

```

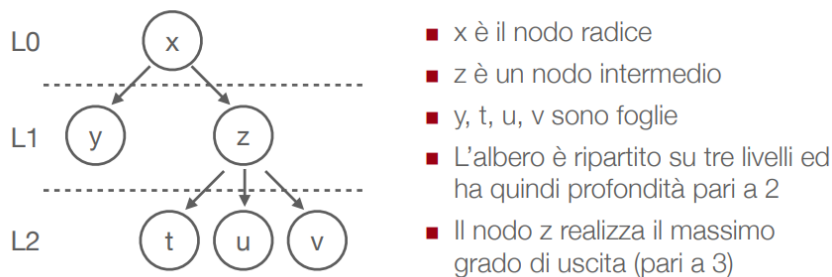
Ora, definiamo l'albero, che è un insieme di elementi (nodi) su cui è definita una relazione di “discendenza” con due proprietà:

- Esiste un unico nodo, detto radice, che non ha predecessori
- Ogni altro nodo ha un unico predecessore

I nodi che non hanno successori sono detti *foglie*.

L'albero è ripartito a livelli e i nodi che non sono né la radice, né una foglia, sono detti nodi *intermedi*.

■ Vediamo un esempio:



Nella caratterizzazione di un albero hanno rilevanza:

- Grado di uscita di un nodo = numero dei suoi successori diretti
- Profondità = distanza di un nodo dalla radice (che ha per definizione profondità 0)

Per “estensione”, la profondità di un albero è data dal max delle profondità dei nodi che lo compongono.

L'insieme dei nodi a stessa profondità forma un livello. Un albero si dice bilanciato quando su qualsiasi nodo la profondità dei sottoalberi differisce al massimo di 1.

Su un albero si eseguono le stesse operazioni che si applicano ad una lista (con un paio di differenze).

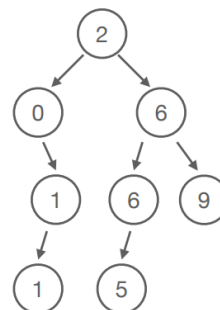
- Su un albero non esiste un unico nodo terminale, ma più foglie.
 - Per cui l'inserimento in coda deve essere qualificato con un criterio di selezione quale foglia è il target dell'operazione
 - L'inserimento in testa (o in posizione intermedia) tende a far degenerare l'albero verso una forma sequenziale; per cui gli inserimenti sono tipicamente eseguiti sulle foglie

Un albero *binario* è un albero sul quale ciascun nodo ha grado di uscita minore o uguale a 2.

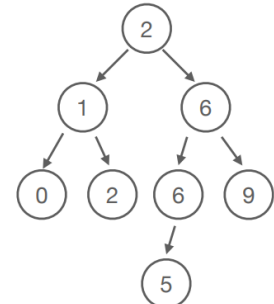
■ Vediamo due esempi:

Un albero binario si dice *di ricerca* quando:

- Il valore codificato da ciascun nodo è maggiore o uguale del valore sul figlio sinistro del nodo stesso, e minore stretto del valore codificato sul figlio destro



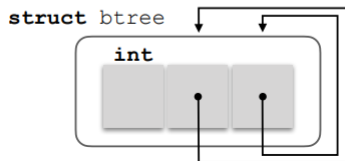
Albero Binario di Ricerca sbilanciato



Albero Binario di Ricerca bilanciato

- Vediamo l'estensione della rappresentazione collegata con puntatori vista per le liste per un albero binario:

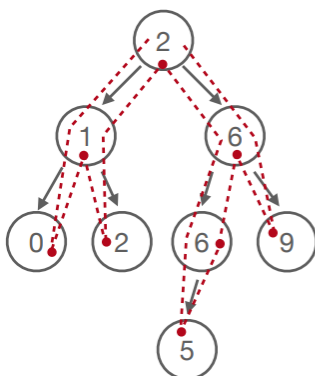
```
struct btree {
    int valore;
    struct btree *leftPtr;
    struct btree *rightPtr;
};
```



La visita di un albero binario di ricerca (ad esempio per funzioni di stampa) può avvenire in tre modi:

- In forma simmetrica (ordinata)
- In forma anticipata (pre-ordine)
- In forma posticipata (post-ordine); in questo caso il nodo viene visitato dopo la sua discendenza (per cui viene comunemente indicata come visita in profondità / "depth-first")

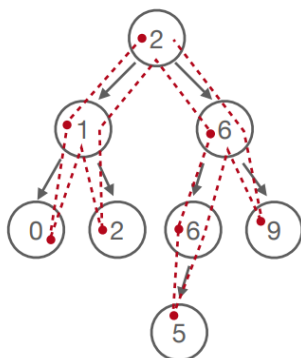
- Vediamo un esempio:



VISITA SIMMETRICA

Output: 0, 1, 2, 2, 5, 6, 6, 9

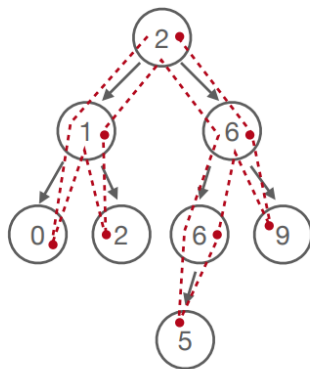
- Vediamo un esempio:



VISITA ANTICIPATA

Output: 2, 1, 0, 2, 6, 6, 5, 9

■ Vediamo un esempio:



VISITA POSTICIPATA

Output: 0, 2, 1, 5, 6, 9, 6, 2

Vediamo quindi alcune funzioni utili:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct btree {
    int valore;
    struct btree *leftPtr;
    struct btree *rightPtr;
};
```

```
typedef struct btree BTree;
```

```
void init(BTree **ptrPtr);
int isempty(BTree *ptr);
int search_rec(BTree *ptr, int val);
void print_rec(BTree *ptr);
void ordinsert_rec(BTree **ptrPtr, int val);
```

```
int main(void) {
    BTree *albero;
    init(&albero);
    printf("Albero vuoto? %d\n", isempty(albero));
```

```
    ordinsert_rec(&albero, 2);
    printf("Albero vuoto? %d\n", isempty(albero));
    ordinsert_rec(&albero, 1);
    ordinsert_rec(&albero, 0);
    ordinsert_rec(&albero, 2);
    ordinsert_rec(&albero, 6);
    ordinsert_rec(&albero, 6);
    ordinsert_rec(&albero, 9);
    ordinsert_rec(&albero, 5);
    print_rec(albero);
```

```
    int target = 7;
    printf("\nL'albero contiene %d? %d\n", target, search_rec(albero, target));
```

Scritto da Gabriel

```

    return 0;
}

void init(BTree **ptrPtr) {
    *ptrPtr = NULL;
}

// restituisce 1 se l'albero è vuoto, 0 altrimenti
int isempty(BTree *ptr) {
    return ptr == NULL;
}

// restituisce 1 se l'albero contie l'elemento val, 0 altrimenti
int search_rec(BTree *ptr, int val) {
    if(ptr != NULL) {
        if(ptr->valore == val) {
            return 1;
        }
        else {
            if(ptr->valore > val)
                search_rec(ptr->leftPtr, val);
            else
                search_rec(ptr->rightPtr, val);
        }
    }
    else {
        return 0;
    }
}

// visita ricorsiva
void print_rec(BTree *ptr) {
    if(ptr!=NULL) {
        //printf("%d ", ptr->valore); // visita in pre-ordine
        print_rec(ptr->leftPtr);
        printf("%d ", ptr->valore); // visita simmetrica (in ordine)
        print_rec(ptr->rightPtr);
        //printf("%d ", ptr->valore); // visita in post-ordine (in profondità / "depth-first")
    }
}

// inserimento in ordine (con ricorsione)
void ordinsert_rec(BTree **ptrPtr, int val) {
    if((*ptrPtr) != NULL) {
        if(val<=(*ptrPtr)->valore)
            ordinsert_rec(&((*ptrPtr)->leftPtr), val);
        else
            ordinsert_rec(&((*ptrPtr)->rightPtr), val);
    }
    else {
        (*ptrPtr) = (BTree*)malloc(sizeof(BTree));
        (*ptrPtr)->valore = val;
    }
}

```

```
    (*ptrPtr)->leftPtr = NULL;  
    (*ptrPtr)->rightPtr = NULL;  
  }  
}
```